

موضوع:

تبیین پذیری و تحلیل زیر ساختار ها

استاد : جناب آقای دکتر موسوی

تدوین کننده : مریم مداح علی و پانیذ آذرنیا

درس: تحلیل شبکه های پیچیده

دانشکده مهندسی برق و کامپیوتر

دانشگاه شیراز

بهمن ماه ۱۴۰۴

فهرست مطالب

مقدمه ۳

توضیح مسئله ۴

معرفی دیتاست ۶

روش حل ۲۳

تحلیل نتایج برای پزشکان و پژوهشگران ۷۸

جمع بندی و نتیجه‌گیری ۸۲

منابع ۸۳

مقدمه

فرآیند توسعه داروهای جدید یکی از پرهزینه‌ترین و زمان‌برترین مسیرهای پژوهشی در علوم پزشکی است. برآوردها نشان می‌دهد که معرفی یک داروی جدید به بازار به طور متوسط ۱۰ تا ۱۵ سال زمان نیاز داشته و هزینه‌ای بالغ بر ۲٫۶ میلیارد دلار صرف آن می‌شود، در حالی که نرخ شکست در مراحل مختلف کارآزمایی‌های بالینی به بیش از ۹۰ درصد می‌رسد [۱]. این محدودیت‌ها سبب شده تا در سال‌های اخیر، توجه بسیاری از پژوهشگران به راهبردهای مکمل و جایگزین مانند بازتوسعه دارو [۱]معطوف شود که در آن کاربردهای درمانی جدید برای ترکیبات دارویی موجود شناسایی می‌گردد.

در همین راستا، پیشرفت فناوری‌های مولکولی در دو دهه اخیر حجم عظیمی از داده‌های زیستی را تولید کرده است. پایگاه‌هایی مانند CTD[2] با گردآوری اطلاعات مستند درباره برهم‌کنش‌های دارو-ژن و ارتباطات ژن-بیماری، منبع ارزشمندی برای مطالعات محاسباتی فراهم آورده‌اند. با این حال، پراکندگی این داده‌ها در منابع گوناگون و ناهمگونی ساختاری آن‌ها، استخراج دانش یکپارچه را با چالش مواجه ساخته است. از سوی دیگر، آنتولوژی‌هایی مانند MONDO با استانداردسازی نام‌گذاری بیماری‌ها و ایجاد ساختاری سلسله‌مراتبی، امکان تلفیق داده‌ها از منابع مختلف را میسر می‌سازند.

روش‌های محاسباتی نوین، به ویژه مدل‌های مبتنی بر گراف، ابزارهای قدرتمندی برای تحلیل داده‌های پیچیده زیستی ارائه داده‌اند. معماری‌هایی مانند شبکه‌های گرافی رابطه‌ای[۳] با قابلیت مدل‌سازی انواع مختلف روابط میان گره‌ها، عملکرد مؤثری در پیش‌بینی ارتباطات جدید در دانش‌گراف‌ها از خود نشان داده‌لند [۲]. با این وجود، کاربرد این مدل‌ها در حوزه پزشکی با چالش توضیح‌پذیری[۴] مواجه است؛ چرا که برای پذیرش نتایج توسط پزشکان و پژوهشگران، لازم است مسیرهای زیستی منجر به هر پیش‌بینی قابل شناسایی و تفسیر باشند.

گزارش حاضر به معرفی یک چارچوب محاسباتی برای تحلیل داده‌های دارو-ژن و ژن-بیماری با هدف شناسایی ارتباطات جدید میان داروهای موجود و بیماری‌های سرطانی می‌پردازد. در ادامه، ابتدا منابع داده و فرآیند آماده‌سازی آن‌ها تشریح می‌شود، سپس معماری مدل پیشنهادی ارائه می‌گردد و در پایان نتایج ارزیابی و خروجی‌های حاصل مورد بحث قرار می‌گیرند.

---

[۱] Drug Repurposing

[2] Comparative Toxicogenomics Database

[۳] Relational Graph Convolutional Networks – R-GCN

[۴] Explainability

توضیح مسئله

سرطان حاصل تجمع تغییرات در ژن‌های تنظیم‌کننده چرخه سلولی، تمایز و مرگ برنامه‌ریزی‌شده سلول‌ها است. این تغییرات عمدتاً در دو گروه ژنی رخ می‌دهند: ژن‌های محرک تومور [۵] که با فعال‌شدگی بیش ازحد، تکثیر سلولی را تحریک می‌کنند و ژن‌های سرکوبگر تومور [۶] که با غیرفعال‌شدن، مهار رشد را از دست می‌دهند.

بسیاری از داروهای مورد استفاده در درمان سرطان از طریق اتصال به محصولات پروتئینی این ژن‌ها یا تأثیر بر اجزای مسیرهای انتقال پیام[۷]وابسته به آن‌ها عمل می‌کنند. برای نمونه، داروی Imatinib با مهار آنزیم کیناز حاصل از جابه‌جایی کروموزومی در لوسمی میلوئید مزمن، رشد سلول‌های سرطانی را متوقف می‌سازد. با وجود چنین نمونه‌های موفقی، در بسیاری از سرطان‌ها چندین مسیر انتقال پیام به طور همزمان دچار اختلال می‌شوند.در سرطان پستان، مسیرهای وابسته به گیرنده استروژن [۸] گیرنده فاکتور رشد اپیدرمی انسانی [۹]و مسیر PI3K/AKT/mTOR ممکن است به تنهایی یا هم‌زمان نقش داشته باشند. این پیچیدگی، تحلیل تک‌ژنی را ناکارآمد ساخته و ضرورت بررسی شبکه‌ای تعاملات مولکولی را آشکار می‌کند. [1]

پایگاه داده CTD[10] با گردآوری برهم‌کنش‌های مستند دارو-ژن و ژن-بیماری مبتنی بر شواهد تجربی، منبع اصلی اطلاعات مورد استفاده در این پروژه است[1]. با این حال، داده‌های خام این پایگاه به دلیل وجود شناسه‌های ناهمگون برای بیماری‌ها مانند OMIM، MeSH، ICD نیازمند استانداردسازی هستند. آنتولوژی MONDO با ارائه ساختاری یکپارچه و روابط سلسله‌مراتبی میان بیماری‌ها، امکان نگاشت این شناسه‌ها به یک سیستم واحد و همچنین فیلتر کردن بیماری‌ها بر اساس حوزه سرطان را فراهم می‌آورد.

در این پروژه، داده‌های حاصل از یکپارچه‌سازی CTD و MONDO برای ساخت یک گراف دانش ناهمگن با سه نوع گره (دارو، ژن، بیماری) و دو نوع یال (دارو-ژن، ژن-بیماری) به کار گرفته می‌شود. مدل شبکه عصبی گرافی رابطه‌ای با بهره‌گیری از مکانیزم‌های به‌اشتراک‌گذاری پارامترها میان انواع

---

[۵] Oncogenes
[۶] Tumor Suppressor Genes
[۷] Signaling Pathways
[۸] Estrogen Receptor
[۹] HER2
[10] Comparative Toxicogenomics Database

روابط، نمایش‌های برداری [11]گره‌ها را یاد گرفته و ارتباطات جدید میان داروهای موجود و بیماری‌های سرطانی را پیش‌بینی می‌کند[۲]. سپس با استفاده از روشGNN Explainer ، زیرگراف‌هایی که بیشترین نقش را در هر پیش‌بینی ایفا کرده‌اند استخراج شده و به عنوان مسیرهای زیستی قابل تفسیر ارائه می‌گردند تا امکان اعتبارسنجی تجربی آن‌ها توسط پژوهشگران فراهم شود.

مدل‌های یادگیری عمیق گرافی، علیرغم قدرت بالا در پیش‌بینی ارتباطات جدید، معمولاً به‌عنوان جعبه‌سیاه عمل می‌کنند. به این معنا که یک امتیاز یا احتمال برای یک جفت دارو-بیماری تولید می‌کنند، اما هیچ توضیحی درباره چرایی آن پیش‌بینی ارائه نمی‌دهند. در کاربردهای بالینی و زیست‌پزشکی، این وضعیت غیرقابل قبول است. یک پزشک یا پژوهشگر نمی‌تواند صرفاً بر اساس یک عدد، درمانی را تجویز کند یا هزینهٔ آزمایش‌های گرانقیمت را توجیه نماید. او نیاز دارد بداند مدل بر اساس کدام شواهد و مسیرهای زیستی به این نتیجه رسیده است.

اینجاست که تشبیه «قاتل و پوشه مدارک» معنادار می‌شود: مدل هوش مصنوعی ما مانند یک کارآگاه است که به یک نتیجه (متهم) رسیده، اما برای اقناع قاضی (پزشک) باید مدارک و شواهد خود را ارائه دهد. این مدارک چیزی نیست جز زیرگرافی از مسیرهای دارو-ژن-بیماری که بیشترین سهم را در تصمیم‌گیری داشته‌اند. به عبارت دیگر، مدل باید بتواند بگوید: «این دارو را برای این بیماری پیشنهاد می‌کنم، زیرا از طریق این ژن‌ها و این روابط با بیماری مرتبط است. »

در این پروژه، ما این هدف را دنبال کرده‌ایم: نه فقط پیش‌بینی ارتباطات جدید، بلکه استخراج و نمایش بصری مسیرهای زیستی منجر به آن پیش‌بینی. خروجی نهایی، یک گراف تعاملی است که به‌صورت شفاف نشان می‌دهد مدل برای هر پیشنهاد دارویی، روی کدام ژن‌ها و کدام روابط تکیه کرده است. این همان «پوشه مدارک» است که به پزشک ارائه می‌شود تا بتواند با اطمینان بیشتری تصمیم بگیرد یا فرضیه‌های جدیدی برای تحقیقات آزمایشگاهی طراحی کند.

[11] Embeddings

معرفی دیتاست

همانطور که اشاره شد ، داده‌های این پروژه از پایگاه CTD استخراج شده است که شامل دو نوع رابطه اصلی است: روابط دارو-ژن (با بیش از ۸ میلیون رکورد اولیه) و روابط ژن-بیماری (با بیش از ۲۹ میلیون رکورد اولیه). با اعمال فیلترهای کیفی مانند انتخاب تنها روابط انسانی، حفظ تنها فرم‌های ژنی (برای دارو-ژن) و روابط با شواهد مستقیم (برای ژن-بیماری)، تعداد روابط به ترتیب به ۴۲۶۶۱ و ۳۴۲۴۷ کاهش یافت. سپس این دو مجموعه بر اساس ژن‌های مشترک با یکدیگر ادغام شدند. برای استانداردسازی نام بیماری‌ها و یکپارچه‌سازی شناسه‌های ناهمگون (عمدتاً MeSH وOMIM) ، از آنتولوژی MONDO استفاده گردید و با بهره‌گیری از ساختار سلسله‌مراتبی آن، تنها بیماری‌های زیرمجموعه سرطان (با ریشه ۰۰۰۴۹۹۲) حفظ شدند. در نهایت، گراف نهایی شامل ۲۶۷۰ گره (۶۴۵ دارو، ۱۹۰۶ ژن و ۱۱۹ بیماری) و ۳۳۵۲۸ یال (شامل دو نوع رابطه دارو-ژن و ژن-بیماری) می‌باشد. جزئیات بیشتردرباره دیتاست ها ، فرآیند استخراج و پالایش داده‌ها در ادامه ارائه شده است.

در این بخش به فرآیند ساخت گراف دانش از داده‌های خام زیست پزشکی پرداخته می‌شود. همانگونه که در معماریDecagon [1] نشان داده شده، ساخت یک گراف چندلایه [۱۲]نیازمند یکپارچه‌سازی داده‌های ناهمگن و استانداردسازی شناسه‌ها است. در این پروژه، نه ماژول پایتون طراحی شده‌اند که هر یک وظیفه مشخصی در این زنجیره بر عهده دارند.

۱. ماژول chem_gene.py: استخراج روابط دارو_ژن

در معماری Decagon [۱]، یکی از لایه‌های اصلی گراف، روابط بین داروها و پروتئین‌ها [۱۳] است که نقش کلیدی در مدل‌سازی اثرات داروها ایفا می‌کند. در این پروژه، به دلیل تمرکز بر مسیرهای دارو ⟶ ژن ⟶ بیماری، روابط دارو-ژن معادل همان لایه در نظر گرفته شده‌اند. پایگاه داده CTD با گردآوری برهم‌کنش‌های مستند بین مواد شیمیایی و ژن‌ها، منبع اصلی استخراج این روابط است. [۱]

ماژول chem_gene.py با هدف پالایش داده‌های خام CTD و استخراج یال‌های معتبر دارو-ژن طراحی شده است. داده‌های خام این پایگاه به صورت فایل فشرده با بیش از ۸ میلیون رکورد ارائه می‌شوند که شامل اطلاعات جانبی مانند نوع تعامل، ایزوفرم‌های ژنی و ارگانیسم‌های مختلف هستند.

[۱۲] multimodal graph
[۱۳] drug-protein target interactions

برای ساخت گراف دانش، تنها برهم‌کنش‌های انسانی با فرم ژنی مورد نیاز هستند، زیرا سایر اشکال مانند ایزوفرم‌ها یا پروتئین‌ها ممکن است منجر به گره‌های تکراری یا مبهم در گراف شوند.

۱.۱.کد ماژول:

```
import pandas as pd
```

در خط اول، کتابخانه pandas وارد می‌شود. این کتابخانه پرکاربردترین ابزار برای پردازش داده‌های جدولی در پایتون است و توابع قدرتمندی برای خواندن، فیلتر کردن و تبدیل داده‌ها ارائه می‌دهد.

```
def get_chem_gene() :
  path = "CTD_chem_gene_ixns.tsv.gz"
```

تابعی به نام get_chem_gene تعریف می‌شود که هیچ ورودی نمی‌گیرد و قرار است داده‌های پردازش‌شده را بازگرداند. در داخل تابع، مسیر فایل فشرده حاوی داده‌های دارو_ژن در متغیر path ذخیره می‌شود.

```
  df = pd.read_csv(
    path,
    sep="\t",
    compression="gzip",
    comment="#",
    header=None,
    low_memory=False
  )
```

تابع read_csv از pandas برای خواندن فایل استفاده می‌کند. پارامتر"sep="\t نشان می‌دهد که فایل با تب جداسازی شده است . "compression="gzip مشخص می‌کند که فایل فشرده است و باید پیش از خواندن از حالت فشرده خارج شود . "comment="# باعث می‌شود خطوطی که با علامت # شروع می‌شوند (معمولاً توضیحات ابتدای فایل) نادیده گرفته شوند .header=None نشان می‌دهد که فایل header ندارد و نام ستون‌ها باید بعداً تعیین شوند .low_memory=False به pandas می‌گوید که کل

فایل را یکباره در حافظه بخواند و از تکه‌تکه کردن داخلی خودداری کند که این کار سرعت پردازش را افزایش می‌دهد.

```
df.columns = [
    "ChemicalID", "ChemicalName", "CasRN", "GeneSymbol",
    "GeneID", "GeneForm", "Organism", "OrganismID",
    "Interaction", "InteractionActions", "PubMedIDs"
]
```

در این خط، نام ستون‌ها به صورت دستی به DataFrame اختصاص داده می‌شود. این نام‌ها بر اساس ساختار مستند فایل CTD انتخاب شده‌اند. ستون‌های مهم برای این پروژه عبارتند از: ChemicalID (شناسه دارو)، GeneID (شناسه ژن)، GeneForm (نوع محصول ژنی) و Organism (نام ارگانیسم).

```
df = df[df["Organism"] == "Homo sapiens"]
```

فیلتر اول: تنها ردیف‌هایی که در ستون Organism مقدار "Homo sapiens" دارند، نگه داشته می‌شوند. این کار تضمین می‌کند که تمام روابط واردشده به گراف مربوط به انسان هستند، زیرا هدف نهایی پروژه تحلیل بیماری‌های انسانی است.

```
df = df[df["GeneForm"] == "gene"]
```

فیلتر دوم: تنها ردیف‌هایی که در ستون GeneForm مقدار "gene" دارند، نگه داشته می‌شوند. در پایگاه CTD، GeneForm می‌تواند مقادیری مانند ""gene"، protein"" یا "transcript" داشته باشد. انتخاب تنها "gene" باعث می‌شود که هر ژن تنها یک بار در گراف ظاهر شود و از ایجاد گره‌های متعدد برای ایزوفرم‌های مختلف یک ژن جلوگیری شود.

```
df = df[["ChemicalID", "GeneID"]].dropna()
```

از بین ستون‌های متعدد، تنها دو ستون ChemicalID و GeneID انتخاب می‌شوند. این دو ستون برای ساخت یال‌های دارو–ژن در گراف کافی هستند. سپس تابع dropna() ردیف‌هایی را که در این دو ستون مقدار تهی [۱۴] دارند، حذف می‌کند.

[۱۴] NaN

```
df = df.drop_duplicates()
```

در نهایت، ردیف‌های تکراری حذف می‌شوند. اگر یک جفت دارو–ژن چندین بار در داده‌ها تکرار شده باشد (مثلاً به دلیل چندین منبع یا چند نوع تعامل)، تنها یک بار در گراف ظاهر خواهد شد. این کار از وزن‌دهی ناخواسته به یال‌ها جلوگیری می‌کند.

```
return df
```

تابع، DataFrame نهایی را بازمی‌گرداند. این DataFrame شامل دو ستون ChemicalID و GeneID است و هر سطر آن نشان‌دهنده یک یال معتبر دارو–ژن می‌باشد.

۱.۲.تفسیر خروجی:

پس از اجرای ماژول، پیام "Dataset created successfully!" در کنسول نمایش داده می‌شود که نشان می‌دهد فرآیند پردازش با موفقیت انجام شده است. سپس تعداد کل ردیف‌های خروجی ۴۲۶۶۱ اعلام می‌شود. این عدد بیانگر تعداد جفت‌های یکتای (دارو_ ژن) انسانی است که پس از اعمال فیلترهای زیر در داده‌های خام باقی مانده‌اند: حذف رکوردهای غیرانسانی، حذف ایزوفرم‌های ژنی (تنها ژن‌های اصلی با GeneForm برابر "gene" نگه داشته شده‌اند)، حذف ردیف‌های دارای مقادیر تهی در شناسه‌ها و حذف رکوردهای تکراری.

در ادامه، پنج ردیف نمونه از داده‌ها چاپ شده است. ستون اول شامل شناسه دارو (ChemicalID) است که در برخی موارد مانند "hydroxynortriptyline-۱۰" به صورت نام شیمیایی و در مواردی مانند "tetrabromoethane-۱’۱’۲’۲" به صورت ترکیب شیمیایی نمایش داده می‌شود. ستون دوم شامل شناسه عددی ژن هدف (GeneID) در پایگاه NCBI است. برای مثال، داروی "۱’۱’۲’۲-tetrabromoethane" در دو ردیف مجزا با دو ژن متفاوت (۸۰۰۴۵ و ۴۰۰۹) ظاهر شده که نشان‌دهنده تعامل این دارو با بیش از یک ژن است.

علاوه بر خروجی کنسول، این ماژول یک فایل CSV با نام CTD_chem_gene_ixns.tsv.csv در همان مسیر ذخیره می‌کند که شامل تمام ۴۲۶۶۱ ردیف خروجی است. این فایل برای استفاده در مراحل بعدی پروژه (ادغام با سایر داده‌ها و ساخت گراف) ذخیره می‌شود، هرچند محتوای آن در خروجی کنسول نمایش داده نمی‌شود.

۲. ماژول gene_disease.py : استخراج روابط ژن ـ بیماری

در ادامه فرآیند ساخت گراف دانش، به سراغ دومین لایه ارتباطی می‌رویم. در معماریDecagon [1] ، برهم‌کنش‌های پروتئین-پروتئین نقش کلیدی در انتشار اطلاعات بین اهداف دارویی ایفا می‌کنند. در پروژه حاضر، با توجه به تمرکز بر مسیر دارو ← ژن ← بیماری، روابط ژن-بیماری معادل عملکردی آن لایه محسوب می‌شوند. به بیان دیگر، اگر در مدل Decagon برهم‌کنش‌های فیزیکی بین پروتئین‌ها امکان مدل‌سازی زنجیره‌های بلندتر را فراهم می‌کنند، در اینجا روابط ژن-بیماری نقش واسط را برای اتصال داروها به بیماری‌ها از طریق ژن‌ها بر عهده دارند.

پایگاه داده CTD یکی از غنی‌ترین منابع در زمینه ارتباطات ژن-بیماری است که اطلاعات آن از دو مسیر اصلی تأمین می‌شود: روابط با شواهد مستقیم [۱۵]که مبتنی بر آزمایش‌های تجربی یا متون علمی تأییدشده هستند، و روابط استنتاجی [۱۶]که از طریق هم‌بیانی ژن‌ها یا شباهت‌های کارکردی به دست آمده‌اند. انتخاب نوع شواهد در کیفیت نهایی گراف تأثیر مستقیم دارد. روابط مستقیم اگرچه تعداد کمتری دارند، اما احتمال خطا در آنها به مراتب کمتر است و پایه‌ای محکم‌تر برای پیش‌بینی‌های بعدی فراهم می‌کنند. به همین دلیل، در این ماژول تنها روابط دارای شواهد مستقیم استخراج می‌شوند.

چالش اصلی در پردازش فایل‌هایCTD ، حجم بسیار بالای داده‌هاست. فایل CTD_genes_diseases.tsv.gzحاوی بیش از ۲۹ میلیون رکورد است که بارگذاری یکباره آن در حافظه، حتی برای سیستم‌های قدرتمند نیز مشکل‌ساز خواهد بود. برای غلبه بر این محدودیت، از روش خواندن تکه‌تکه [۱۷]استفاده می‌شود که در ادامه کد و شرح آن ارائه می‌گردد.

۲.۱. کد ماژول:

```
def get_gene_disease():
    path = "CTD_genes_diseases.tsv.gz"
    chunksize = 200_000
    edges = []
```

[۱۵] Direct Evidence
[۱۶] Inferred
[۱۷] chunking

در خط اول، مسیر فایل فشرده حاوی داده‌های ژن-بیماری در متغیرpath ذخیره می‌شود. سپس اندازه هر تکه ۲۰۰ هزار رکورد تعیین می‌گردد - این عدد حاصل توازن بین مصرف حافظه و تعداد دفعات خواندن فایل است. انتخاب این مقدار تضمین می‌کند که حتی روی سیستم‌هایی با حافظه محدود (مثلاً ۸ گیگابایت) نیز برنامه بدون خطای کمبود حافظه اجرا شود. لیست خالیedges برای جمع‌آوری تکه‌های پردازش‌شده در نظر گرفته می‌شود.

```
for chunk in pd.read_csv(
path,
sep="\t",
compression="gzip",
comment="#",
header=None,
chunksize=chunksize,
low_memory=False
):
```

حلقه for روی تکه‌های فایل می‌چرخد. تابع read_csv با پارامتر chunksize به جای بازگرداندن یک DataFrameکامل، یک شیء تکرارشونده از تکه‌ها را تولید می‌کند. سایر پارامترها مشابه ماژول قبلی هستند: جداکننده تب(sep="\t") ، فشرده‌سازیgzip ، نادیده گرفتن خطوط توضیح(comment="#") ، و عدم وجود هدر (header=None) . پارامتر low_memory=False نیز تضمین می‌کند که نوع داده‌ها در کل فایل به درستی تشخیص داده شود.

```
chunk.columns = [
"GeneSymbol",
"GeneID",
"DiseaseName",
"DiseaseID",
"DirectEvidence",
"InferenceGeneSymbol",
```

```
        "InferenceScore",
        "OmimIDs",
        "PubMedIDs"
    ]
```

پس از خواندن هر تکه، نام ستون‌ها بر اساس ساختار مستند فایل CTD اختصاص داده می‌شود. ستون‌های کلیدی عبارتند از: GeneID (شناسه عددی ژن در پایگاه NCBI)، GeneSymbol (نماد استاندارد ژن)، DiseaseID (شناسه بیماری در منابعی مانند MeSH یا OMIM)، DiseaseName (نام بیماری)، و DirectEvidence (نشان‌دهنده وجود شواهد مستقیم).

```
chunk = chunk[chunk["DirectEvidence"].notna()]
```

این خط مهم‌ترین فیلتر کیفی را اعمال می‌کند: تنها ردیف‌هایی نگه داشته می‌شوند که در ستون شواهد مستقیم مقدار داشته باشند (یعنی notna). وجود مقدار در این ستون به معنای آن است که ارتباط ژن-بیماری مورد نظر با شواهد تجربی یا متون علمی تأیید شده است. این فیلتر تعداد رکوردها را به شدت کاهش می‌دهد اما کیفیت داده‌ها را تضمین می‌کند.

```
chunk = chunk[chunk["GeneID"].notna()]
```

برخی از ردیف‌ها ممکن است فاقد شناسه ژن باشند که برای ساخت گراف غیرقابل استفاده هستند. این خط چنین ردیف‌هایی را حذف می‌کند.

```
    chunk = chunk[["GeneID", "GeneSymbol", "DiseaseID", "DiseaseName"]]
    edges.append(chunk)
```

از میان ستون‌های متعدد، تنها چهار ستون مورد نیاز انتخاب می‌شوند: شناسه ژن، نماد ژن، شناسه بیماری و نام بیماری. سپس تکه پردازش‌شده به لیست یال ها اضافه می‌گردد.

```
    df = pd.concat(edges).drop_duplicates()
    return df
```

پس از اتمام حلقه، تمام تکه‌های ذخیره‌شده با pd.concat در یک DataFrame ادغام می‌شوند. سپس ردیف‌های تکراری حذف می‌گردند تا هر جفت (ژن، بیماری) تنها یک بار در خروجی ظاهر شود. در نهایت DataFrame نهایی بازگردانده می‌شود.

۲.۲. تفسیر خروجی:

پس از اجرای ماژول، پیام‌های زیر در کنسول نمایش داده می‌شود:

Gene-Disease dataset created successfully!

Number of edges: 34247

	GeneID	GeneSymbol	DiseaseID	DiseaseName
14171	۵۰۵۱۸	A	MESH:D003872	Dermatitis
14190	۵۰۵۱۸	A	MESH:D003920	Diabetes Mellitus
14201	۵۰۵۱۸	A	MESH:D003924	Diabetes Mellitus, Type 2
14217	۵۰۵۱۸	A	MESH:D003928	Diabetic Nephropathies
14278	۵۰۵۱۸	A	MESH:D004487	Edema

تعداد ۳۴۲۴۷ یال (ردیف) پس از اعمال فیلترهای شواهد مستقیم و حذف رکوردهای فاقد شناسه ژن به دست آمده است. این عدد در مقایسه با حجم اولیه فایل (بیش از ۲۹ میلیون رکورد) کاهش چشمگیری دارد و نشان‌دهنده گزینش دقیق تنها ارتباطات با کیفیت بالا است.

نمونه نمایش‌داده‌شده شامل پنج ردیف اول از خروجی نهایی است:

- ستون GeneID شناسه عددی ژن در پایگاه NCBI را نشان می‌دهد (مثلاً ۵۰۵۱۸).
- ستون GeneSymbol نماد استاندارد ژن است . در این نمونه همه ردیف‌ها دارای نماد "A"هستند که احتمالاً کوتاه‌شده نام ژن است؛ اما در داده‌های واقعی ممکن است نمادهای متنوعی دیده شود.
- ستون DiseaseID شناسه بیماری در منابعی مانند MeSH را شامل می‌شود (مثلاً MESH:D003872).
- ستون DiseaseName نام بیماری مربوطه را نشان می‌دهد(مانندDermatitis ، Diabetes، Mellitus و غیره) .

نکته قابل توجه در این خروجی آن است که یک ژن واحد (با شناسه ۵۰۵۱۸) با چندین بیماری مختلف مرتبط است. این پدیده که در داده‌های زیست‌پزشکی بسیار رایج است، بیانگر نقش

چندکارکردی بسیاری از ژن‌ها در مسیرهای مختلف بیماری‌زایی است و ضرورت مدل‌سازی شبکه‌ای را بیش از پیش آشکار می‌کند.

علاوه بر نمایش کنسول، ماژول فایل gene_disease_edges.csv را در همان مسیر ذخیره می‌کند که شامل تمام ۳۴۲۴۷ ردیف و چهار ستون یادشده است. این فایل در مرحله بعدی (ادغام با داده‌های دارو-ژن) به کار گرفته خواهد شد.

۳. ماژول merge_data.py : یکپارچه‌سازی داده‌ها و فیلتر کردن بیماری‌های سرطانی

پس از استخراج روابط دارو-ژن و ژن-بیماری، گام بعدی یکپارچه‌سازی این دو مجموعه داده در یک ساختار واحد است. در معماری Decagon [۱]، یک گراف چندلایه با دو نوع گره (دارو و پروتئین) و چندین نوع یال ساخته می‌شود. در پروژه حاضر، با اضافه شدن گره‌های بیماری، گراف به سه نوع گره (دارو، ژن، بیماری) و دو نوع یال (دارو-ژن و ژن-بیماری) توسعه می‌یابد. با این حال، داده‌های خام پایگاه CTD دارای شناسه‌های ناهمگون برای بیماری‌ها هستند که مستقیماً قابل استفاده در یک گراف یکپارچه نیستند. برای مثال، یک بیماری ممکن است در یک منبع با شناسه MeSH، در منبع دیگر با OMIM و در سومی با ICD کدگذاری شده باشد. این ناهمگونی، مدل‌سازی یکپارچه را با چالش مواجه می‌کند.

آنتولوژی MONDO [۲] با هدف استانداردسازی نام‌گذاری بیماری‌ها و ایجاد روابط سلسله‌مراتبی میان آنها توسعه یافته است. این آنتولوژی، شناسه‌های منابع مختلف (از جمله MeSH، OMIM، ICD و ...) را به یک شناسه یکتای MONDO نگاشت می‌کند و همچنین ساختار والد-فرزندی میان بیماری‌ها را تعریف می‌نماید. بهره‌گیری از MONDO دو مزیت عمده برای این پروژه دارد: نخست، امکان یکپارچه‌سازی داده‌های حاصل از منابع گوناگون را فراهم می‌آورد و دوم، با استفاده از درخت سلسله‌مراتبی آن می‌توان بیماری‌ها را بر اساس حوزه‌های بالینی (مانند سرطان) فیلتر کرد. با توجه به هدف پروژه که بازتوسعه داروهای سرطان است، تنها بیماری‌های بدخیم باید در گراف نهایی حضور داشته باشند.

ماژول merge_data.py با هدف یکپارچه‌سازی داده‌های دارو-ژن و ژن-بیماری، نگاشت شناسه‌های بیماری به MONDO و سپس فیلتر کردن بیماری‌های سرطانی طراحی شده است. این ماژول از دو

فایل جانبی که از وب‌سایت MONDO دریافت شده‌اند استفاده می‌کند: mondo_references.csv (شامل نگاشت شناسه‌های خارجی به MONDO) و mondo_parents.csv (شامل روابط والد-فرزندی بین نودهای MONDO). در ادامه، کد این ماژول خط به خط بررسی می‌شود.

۳.۱.کد ماژول:

```
from chem_gene import *
from gene_disease import *

import pandas as pd
from collections import defaultdict, deque
CANCER_ROOT = "0004992"
```

در خطوط ابتدایی، توابع کمکی از دو ماژول پیشین (chem_gene و gene_disease)وارد می‌شوند. کتابخانه‌های pandas،defaultdict وdeque برای پردازش داده‌ها و پیمایش گراف سلسله‌مراتبی MONDOفراخوانی می‌شوند. ثابتCANCER_ROOT با مقدار "0004992"تعریف می‌شود که شناسه ریشه سرطان در آنتولوژی MONDO است.

```
def merge():
    ceg = get_chem_gene()
    cgd = get_gene_disease()
```

در تابع merge، ابتدا داده‌های دارو-ژن و ژن-بیماری با فراخوانی توابع مربوطه بارگذاری می‌شوند. هر دو تابع داده‌ها را به صورت DataFrameپانداس بازمی‌گردانند.

```
    ceg["GeneID"] = ceg["GeneID"].astype(str)
    cgd["GeneID"] = cgd["GeneID"].astype(str)
```

از آنجا که شناسه‌های ژن در پایگاه CTD به صورت عددی ذخیره شده‌اند اما در فرآیند ادغام ممکن است تطابق نوع داده مشکل‌ساز شود، هر دو ستونGeneID به رشته تبدیل می‌شوند.

```
    ceps_raw = ceg.merge(cgd, on="GeneID", how="inner")
    print("After Gene merge:", ceps_raw.shape)
```

دو DataFrame بر اساس ستون مشترک GeneID با یکدیگر ادغام می‌شوند. نوع ادغام inner به این معناست که تنها ردیف‌هایی باقی می‌مانند که ژن آنها هم در مجموعه دارو-ژن و هم در مجموعه ژن-بیماری حضور داشته باشد. خروجی این مرحله در کنسول چاپ می‌شود: After Gene merge: (142254, 5) . عدد ۱۴۲۲۵۴ تعداد جفت‌های (دارو، ژن، بیماری) حاصل از ادغام است و ۵ ستون شامل شناسه دارو، شناسه ژن، نماد ژن، شناسه بیماری و نام بیماری می‌باشند.

```
if ceps_raw.empty:
    print("WARNING: CEPS raw is empty after Gene merge")
    return ceps_raw
```

در صورت خالی بودن نتیجه، یک هشدار چاپ شده و تابع متوقف می‌شود. در این اجرا داده خالی نیست.

```
mask = ceps_raw["DiseaseID"].astype(str).str.match(r"^(MESH:|OMIM:)")
ceps_raw = ceps_raw[mask].copy()
ceps_raw["DiseaseID_norm"] = ceps_raw["DiseaseID"]
```

در این مرحله، مجموعه‌داده‌ی اولیه CEPS به‌گونه‌ای فیلتر می‌شود که تنها بیماری‌هایی با شناسه‌های مبتنی بر آنتولوژی‌های OMIM و MESH حفظ شوند(زیرا فایل gene_disease_edges.csv فقط دارای آنتولوژی‌های OMIM و MESH است). این کار با استفاده از یک عبارت باقاعده انجام می‌شود که ابتدای رشته‌ی DiseaseID را بررسی کرده و فقط رکوردهایی را که با پیشوند :MESH یا :OMIM شروع می‌شوند انتخاب می‌کند.

پس از اعمال فیلتر، یک ستون جدید با نام DiseaseID_norm ایجاد می‌شود که در این نسخه، شناسه بیماری به‌صورت کامل و همراه با پیشوند آنتولوژی (برای مثال MESH:D003872 یا OMIM:114480) نگه داشته می‌شود. این طراحی باعث می‌شود که شناسه‌ها به‌صورت یکتا و بدون ابهام آنتولوژیکی در مراحل بعدی ادغام مورد استفاده قرار گیرند.

```
mondo_xref = pd.read_csv("mondo/mondo_references.csv", dtype=str)
```

فایل mondo_references.csv که حاوی نگاشت بین شناسه‌های خارجی و شناسه‌های MONDO است خوانده می‌شود. پارامتر dtype=str تضمین می‌کند که تمام ستون‌ها به صورت رشته بارگذاری شوند تا از مشکلات تبدیل عددی جلوگیری شود.

```
mondo_xref = mondo_xref [
        mondo_xref["ontology"].isin(["OMIM", "MESH"])
][["ontology", "ontology_id", "mondo_id"]]
```

از جدول مرجع MONDO ، تنها رکوردهایی انتخاب می‌شـــوند که متعلق به آنتولوژی‌های OMIM یا MESH باشـند. سـپس فقط سـتون‌های مرتبط شـامل نوع آنتولوژی(ontology) ، شـناسـه‌ی بیماری در آنتولوژی مربوطه (ontology_id) و شناسه‌ی متناظر در MONDO (mondo_id) نگه داشته میشوند.

```
mondo_xref["DiseaseID_norm"] = (
        mondo_xref["ontology"] + ":" + mondo_xref["ontology_id"]
)
```

برای همسان‌سازی فرمت شناسه‌ها با داده‌های CEPS ، یک ستون جدید با نام DiseaseID_norm ساخته می‌شود که از ترکیب نام آنتولوژی و شناسه‌ی آن تشکیل شده است (برای مثال OMIM:114480 یا MESH:D003872). این فرمت دقیقاً با قالب شناسه‌های موجود در ceps_raw مطابقت دارد و امکان ادغام مستقیم دو جدول را فراهم می‌کند.

```
mondo_xref = mondo_xref[["DiseaseID_norm", "mondo_id"]]
mondo_xref.columns = ["DiseaseID_norm", "MONDO"]
```

در نهایـت، تنهـا سـتون‌های مـورد نیـاز بـرای ادغـام نگـه داشـته می‌شـوند و نـام سـتون mondo_id بـه MONDO تغییـر می‌یابـد تـا معنـای آن بـه‌عنوان شناسـه‌ی اسـتاندارد بیمـاری در آنتولـوژی MONDO شفاف‌تر باشد. ایـن جـدول نهـایی (mondo_xref) بـه‌عنوان مرجـع نگاشـت بـرای اتصـال داده‌هـای CEPS بـه فضای مفهومی MONDO مورد استفاده قرار می‌گیرد .

```
ceps = ceps_raw.merge(
    mondo_xref,
    on="DiseaseID_norm",
    how="left"
)
```

دیتافریم اصلی با دیتافریم مرجع MONDO بر اساس ستون DiseaseID_norm به صورت left join ادغام می‌شود. با این کار، ستون MONDO به رکوردها اضافه می‌گردد. رکوردهایی که شناسه بیماری آنها در مرجع یافت نشود، در این ستون مقدار NaNخواهند داشت.

```
print("Before MONDO drop:", ceps.shape)
print("Mapped MONDO count:", ceps["MONDO"].notna().sum())
```

تعداد کل ردیف‌ها و تعداد ردیف‌هایی که موفق به دریافت شناسه MONDO شده‌اند چاپ می‌شود : (Before MONDO drop: (146852, 7 و Mapped MONDO count: 106278.تعداد ۱۰۶۲۷۸ ردیف معادل حدود ۷۲٪ از کل داده‌ها دارای نگاشت معتبر به MONDO هستند. علت افزایش تعداد ردیف‌ها نسبت به مرحله قبل (۱۴۲۲۵۴) این است که درleft join ، ردیف‌های بدون تطابق نیز حفظ می‌شوند و تعداد ستون‌ها به ۷ افزایش یافته است.

```
unmapped = ceps[ceps["MONDO"].isna()][["DiseaseID", "DiseaseID_norm"]].drop_duplicates()
if not unmapped.empty:
    unmapped.to_csv("unmapped_diseases.csv", index=False)
    print(f"Unmapped diseases saved: {len(unmapped)}")
```

ردیف‌هایی که فاقد MONDO هستند (حدود ۴۰۵۷۴ ردیف) استخراج شده و پس از حذف تکراری‌ها (۹۳۲ بیماری منحصربه‌فرد) در فایل unmapped_diseases.csvذخیره می‌شوند. این فایل برای آنالیز بهتر مفید است.

```
ceps = ceps.dropna(subset=["MONDO"])
if ceps.empty:
    print("WARNING: All rows dropped after MONDO mapping")
    return ceps
```

ردیف‌های فاقد MONDO حذف می‌شوند. اگر پس از این حذف دیتافریم خالی شود، تابع متوقف می‌گردد.

```
parents = pd.read_csv("mondo/mondo_parents.csv", dtype=str)
```

فایل mondo_parents.csv که شامل روابط والد–فرزندی بین نودهای MONDO است خوانده می‌شود.

```
parents = parents[
    parents["parent"].str.fullmatch(r"\d+") &
    parents["child"].str.fullmatch(r"\d+")
]
```

برای اطمینان از اینکه والد و فرزند هر دو شناسه‌های عددی معتبر هستند، از fullmatch با الگوی رقمی استفاده می‌شود. رکوردهایی که حاوی کاراکتر غیرعددی باشند حذف می‌شوند.

```
graph = defaultdict(list)
for _, row in parents.iterrows():
    graph[row["parent"]].append(row["child"])
```

یک گراف جهت‌دار با استفاده از defaultdict ساخته می‌شود. برای هر ردیف، فرزند به لیست همسایگان والد اضافه می‌شود.

```
print("Children of root:", graph.get(CANCER_ROOT, [])[:10])
```

برای اطمینان از صحت گراف، ده فرزند مستقیم ریشه سرطان چاپ می‌شود:

```
['0000376', '0000621', '0000637', ...]
```

```
def get_descendants(root):
    seen = set()
    q = deque([root])
    while q:
        node = q.popleft()
        for child in graph.get(node, []):
            if child not in seen:
                seen.add(child)
                q.append(child)
    return seen
```

تابع get_descendants با استفاده از جستجوی عرضی(BFS) ، تمام نوادگان یک ریشه را استخراج می‌کند. این تابع مجموعه‌ای از شناسه‌های MONDO را بازمی‌گرداند.

```
cancer_mondos = get_descendants(CANCER_ROOT)
cancer_mondos.add(CANCER_ROOT)
```

ابتدا تمام نوادگان ریشه سرطان استخراج می‌شوند (۲۲۹۲ نود) و سپس خود ریشه نیز به مجموعه اضافه می‌گردد تا در مجموع ۲۲۹۳ نود به دست آید. پیام Cancer set size: 2293 در خروجی تأیید این مرحله است.

```
print("Final CEPS shape:", ceps.shape)
print("Unique cancer MONDOs:", ceps["MONDO"].nunique())
```

پیش از اعمال فیلتر سرطان، تعداد کل ردیف‌ها (۱۰۶۲۷۸) و تعدادMONDO های منحصربه‌فرد موجود در داده‌ها (۴۵۲۸) چاپ می‌شود.

```
ceps = ceps[ceps["MONDO"].isin(cancer_mondos)]
```

تنها ردیف‌هایی که شناسه MONDO آنها در مجموعه سرطان (۲۲۹۳ نود) قرار دارد نگه داشته می‌شوند.

```
print("Final CEPS shape after filter:", ceps.shape)
print("Unique cancer MONDOs after filter:", ceps["MONDO"].nunique())
```

پس از فیلتر، تعداد ردیف‌ها به ۱۶۷۶۴ کاهش یافته و تعدادMONDO های منحصربه‌فرد به ۱۳۴ می‌رسد. این کاهش شدید نشان می‌دهد که از میان بیش از ۴۵۰۰ بیماری موجود در داده‌ها، تنها ۱۳۴ بیماری در زیردرخت سرطان قرار دارند. این گزینش دقیق، تمرکز مدل بر بازتوسعه داروهای سرطان را تضمین می‌کند.

```
if not ceps.empty:
    print("Sample Disease Names (first 20 unique):")
    print(ceps["DiseaseName"].unique()[:20])
else:
    print("No disease names to display (empty dataset)")
```

در نهایت، ۲۰ نام بیماری منحصربه‌فرد از داده‌های نهایی نمونه‌گیری و چاپ می‌شود. همان‌طور که در خروجی مشاهده می‌شود، نام‌هایی مانند Carcinoma، Leukemia, Myeloid, Acute، Multiple Myelomaو ... همگی مربوط به انواع سرطان هستند.

```
    return ceps
```

تابع، دیتافریم نهایی را بازمی‌گرداند.

```
if __name__ == "__main__":
    df = merge()
    print("Merge completed.")
    print("Final dataset size:", df.shape)
```

در بخش اصلی برنامه، تابع merge فراخوانی شده و نتایج نهایی چاپ می‌شود : Final dataset size: (16764, 7) . این دیتافریم شامل ۱۶۷۶۴ ردیف و ۷ ستون است که هر ردیف نمایانگر یک رابطه دارو-ژن-بیماری با شناسه‌های استاندارد شده MONDO می‌باشد.

۳.۲. تفسیر خروجی:

خروجی کنسول ماژول merge_data.py به صورت زیر است:

```
After Gene merge: (142254, 5)
Before MONDO drop: (146848, 7)
Mapped MONDO count: 106274
Unmapped diseases saved: 932
Children of cancer root: ['0000376', '0000621', '0000637', '0000653', '0002087', '0002100',
'0002149', '0002402', '0002516', '0002813']
Cancer set size: 2293
Unique MONDOs before filter: 4528
Final CEPS shape after filter: (16764, 7)
Unique cancer MONDOs: 134
Sample Disease Names (first 20 unique):
['Carcinoma', 'Carcinoma, Hepatocellular', 'Leukemia, Myeloid, Acute', ...]
Merge completed.
Final dataset size: (16764, 7)
```

این اعداد نمایانگر فرآیند پالایش چندلایه داده‌ها هستند:

از ۱۴۲۲۵۴ جفت (دارو، ژن، بیماری) اولیه، پس از نگاشت بهMONDO ، ۱۰۶۲۷۴ ردیف موفق به دریافت شناسه معتبر می‌شوند.

۹۳۲ بیماری منحصربه‌فرد که قابل نگاشت نبوده‌اند در فایل جداگانه ذخیره می‌شوند.

با استفاده از آنتولوژی MONDO و استخراج زیردرخت سرطان (۲۲۹۳ نود)، تعداد ردیف‌ها به ۱۶۷۶۴ کاهش می‌یابد.

در داده‌های نهایی، ۱۳۴ بیماری سرطانی منحصربه‌فرد حضور دارند که با نمونه‌گیری از نام آنها، تنوع سرطان‌ها (کارسینوم، لوسمی، لنفوم، میلوم و ...) قابل مشاهده است.

دیتافریم نهایی با ۷ ستون (شامل شناسه دارو، شناسه ژن، نماد ژن، شناسه بیماری اصلی، نام بیماری، شناسه نرمال‌شده بیماری و شناسهMONDO)آماده استفاده در مرحله بعدی یعنی ساخت گراف ناهمگن خواهد بود. همچنین فایل csv آن نیز ذخیره می شود . این فرآیند دقیق و چندمرحله‌ای تضمین می‌کند که گراف نهایی تنها شامل ارتباطات با کیفیت بالا و مرتبط با حوزه سرطان باشد و از نویز و داده‌های نامرتبط پالایش یافته است.

روش حل

برای حل مسئله بازتوسعه داروهای سرطان، ابتدا گراف دانش ناهمگنی شامل سه نوع گره (دارو، ژن، بیماری) و دو نوع رابطه (دارو-ژن و ژن-بیماری) از داده‌های پالایش‌شده ساخته شد. سپس با استفاده از معماری شبکه عصبی گرافی رابطه‌ای(R-GCN) ، بازنمایی‌های نهفته هر گره به‌گونه‌ای یاد گرفته شد که بتواند ارتباطات موجود در گراف را حفظ کند. در مرحله بعد، با ترکیب این بازنمایی‌ها و یک رمزگشایDistMult ، امکان پیش‌بینی یال‌های جدید بین داروها و بیماری‌ها فراهم گردید. برای ایجاد قابلیت توضیح‌پذیری، از روش GNN Explainer استفاده شد که با یادگیری ماسک اهمیت یال‌ها برای یک جفت دارو-بیماری مشخص، زیرگراف‌های مؤثر در تصمیم‌گیری مدل را شناسایی می‌کند. در نهایت، با شمارش مسیرهای دارو-ژن-بیماری که از گره‌های مهم عبور می‌کنند، فهرست اولویت‌بندی‌شده‌ای از مسیرهای زیستی مرتبط با هر پیش‌بینی ارائه می‌شود. جزئیات هر یک از این مراحل در ادامه شرح داده شده است.

۱. ماژول hetero_graph.py :ساخت گراف ناهمگن از داده‌های یکپارچه

پس از یکپارچه‌سازی داده‌ها و فیلتر کردن بیماری‌های سرطانی در ماژول merge_data.py، اکنون باید این داده‌ها را به ساختاری تبدیل کنیم که برای مدل‌سازی با شبکه‌های عصبی گرافی قابل استفاده باشد. در معماری Decagon [1]، گراف به صورت چندلایه [۱۸]با دو نوع گره (دارو و پروتئین) و چندین نوع یال (برهم‌کنش پروتئین-پروتئین، هدف‌گیری دارو-پروتئین و عوارض دارو-دارو) مدل‌سازی شده است. در پروژه حاضر، با توجه به منابع داده در دسترس، همانطور که پیش تر اشاره شد ، گرافی با سه نوع گره (دارو، ژن، بیماری) و دو نوع یال (دارو-ژن و ژن-بیماری) طراحی می‌شود. اگرچه در مدل Decagon از یال‌های متعدد با انواع مختلف استفاده شده، اما رویکرد کلی ساخت گراف و تبدیل آن به نمایش یکپارچه برای شبکه‌های عصبی گرافی مشابه است.

[۱۸] multimodal

ماژول hetero_graph.py وظیفه دارد دیتافریم نهایی حاصل از merge_data.py را دریافت کرده و یک گراف ناهمگن [19] با قابلیت ذخیره‌سازی برای مراحل بعدی ایجاد کند. این گراف سپس به فرمت همگن [20] تبدیل می‌شود تا بتوان از لایه‌های استاندارد RGCNConv در کتابخانه PyTorch Geometric استفاده نمود. دلیل این تبدیل آن است که پیاده‌سازی موجود در پروژه(برگرفته از معماری R-GCN [2]) بر روی گراف‌های همگن با edge_type عمل می‌کند.

۱.۱. کد ماژول:

```
import torch
from torch_geometric.data import HeteroData
from merge_data import merge
```

کتابخانه torch برای عملیات تنسوری و HeteroData از torch_geometric.data برای ساخت گراف ناهمگن وارد می‌شوند. تابع merge از ماژول merge_data فراخوانی می‌گردد تا داده‌های یکپارچه شده را دریافت کند.

```
def graph_hetero():
    ceps_df = merge()
```

تابع graph_hetero تعریف می‌شود. ابتدا با فراخوانی ()merge، دیتافریم نهایی (شامل ۱۶۷۶۴ ردیف و ۷ ستون) بارگذاری می‌گردد. توجه کنید که ()merge خود تابعی است که در ماژول قبلی تعریف شده و داده‌های پالایش‌شده را بازمی‌گرداند.

```
    if ceps_df.empty:
        print("WARNING: Input dataset is empty. Graph not created.")
        return None
```

اگر دیتافریم خالی باشد (که در این اجرا نیست)، یک هشدار چاپ شده و تابع متوقف می‌شود.

```
    print("Input dataset shape:", ceps_df.shape)
```

ابعاد دیتافریم ورودی چاپ می‌شود: Input dataset shape: (16764, 7)

---

[19] Heterogeneous Graph
[20] Homogeneous

```
ceps_df["ChemicalID"] = ceps_df["ChemicalID"].astype(str)
ceps_df["GeneID"] = ceps_df["GeneID"].astype(str)
ceps_df["MONDO"] = ceps_df["MONDO"].astype(int)
```

برای اطمینان از یکنواختی نوع داده‌ها، ستون‌های ChemicalID و GeneID به رشته و ستون MONDO به عدد صحیح تبدیل می‌شوند. تبدیل MONDO به عدد صحیح به این دلیل است که این شناسه‌ها در آنتولوژی MONDO به صورت عددی (بدون پیشوند) ذخیره شده‌اند.

```
chemical_ids = ceps_df["ChemicalID"].unique()
gene_ids = ceps_df["GeneID"].unique()
disease_ids = ceps_df["MONDO"].unique()
```

لیست یکتای شناسه‌های داروها، ژن‌ها و بیماری‌ها استخراج می‌شود. این لیست‌ها برای ساخت نگاشت[۲۱] از شناسه اصلی به ایندکس عددی در گراف به کار می‌روند.

```
chemical_id_to_name = {c: c for c in chemical_ids}
```

برای داروها، از آنجا که در داده‌های CTD نام دارو ذخیره نشده (فقط شناسه وجود دارد)، نگاشت شناسه به نام معادل خود شناسه در نظر گرفته می‌شود. در صورت وجود ستون نام دارو، می‌توان آن را جایگزین کرد.

```
if 'GeneSymbol' in ceps_df.columns:
    gene_id_to_name = ceps_df.groupby('GeneID')['GeneSymbol'].first().to_dict()
else:
    gene_id_to_name = {g: g for g in gene_ids}
```

برای ژن‌ها، اگر ستون GeneSymbol موجود باشد (که در داده‌های ما هست)، یک دیکشنری از اولین نماد ژن متناظر با هر GeneID ساخته می‌شود. در غیر این صورت، خود شناسه به عنوان نام در نظر گرفته می‌شود.

[۲۱] mapping

```
if 'DiseaseName' in ceps_df.columns:
    disease_id_to_name = ceps_df.groupby('MONDO')['DiseaseName'].first().to_dict()
else:
    disease_id_to_name = {d: str(d) for d in disease_ids}
```

برای بیماری‌ها نیز مشابه، نام بیماری از ستون DiseaseName استخراج و به هر شناسه MONDO نسبت داده می‌شود.

```
chemical2idx = {c: i for i, c in enumerate(chemical_ids)}
gene2idx = {g: i for i, g in enumerate(gene_ids)}
disease2idx = {d: i for i, d in enumerate  (disease_ids)}
```

سه دیکشنری برای نگاشت شناسه‌های اصلی به ایندکس‌های عددی در گراف ساخته می‌شود. این ایندکس‌ها در بازه‌های مجزا برای هر نوع گره قرار می‌گیرند: داروها از ۰ تا (تعداد داروها -۱)، ژن‌ها از ۰ تا (تعداد ژن‌ها -۱) و بیماری‌ها نیز به همین ترتیب.

```
chem_gene_src = torch.tensor(
    ceps_df["ChemicalID"].map(chemical2idx).values, dtype=torch.long
)
chem_gene_dst = torch.tensor(
    ceps_df["GeneID"].map(gene2idx).values, dtype=torch.long
)
```

برای ساخت یال‌های دارو-ژن، ستون ChemicalID در دیتافریم با استفاده از chemical2idx به ایندکس‌های متناظر نگاشت شده و به تنسور تبدیل می‌شود. این کار برای مبدأ (دارو) و مقصد (ژن) انجام می‌شود. توجه کنید که map پانداس مقادیر را به ایندکس تبدیل می‌کند و values آرایه numpyحاصل را برمی‌گرداند.

```
gene_disease_src = torch.tensor(
```

```
    ceps_df["GeneID"].map(gene2idx).values, dtype=torch.long
)
gene_disease_dst = torch.tensor(
    ceps_df["MONDO"].map(disease2idx).values, dtype=torch.long
)
```

به طریق مشابه، یال‌های ژن–بیماری از ستون‌های GeneID و MONDO ساخته می‌شوند.

```
data = HeteroData()
data["chemical"].num_nodes = len(chemical_ids)
data["gene"].num_nodes = len(gene_ids)
data["disease"].num_nodes = len(disease_ids)
```

یک شیء HeteroData از PyTorch Geometric ساخته می‌شود. این شیء قادر است برای هر نوع گره، ویژگی‌ها و برای هر نوع یال، لیست یال‌ها را ذخیره کند. در اینجا تنها تعداد گره‌ها برای هر نوع مشخص می‌شود (ویژگی‌های اولیه بعداً اضافه خواهند شد).

```
data[("chemical", "chemical_gene", "gene")].edge_index = torch.stack(
    [chem_gene_src, chem_gene_dst]
)
data[("gene", "gene_disease", "disease")].edge_index = torch.stack(
    [gene_disease_src, gene_disease_dst]
)
```

یال‌ها با تعریف سه‌تایی (نوع مبدأ، نوع رابطه، نوع مقصد) به گراف اضافه می‌شوندفرمول عملیات torch.stack به صورت زیر است:

$$\text{stack}([a, b]) = \begin{bmatrix} a_1 & a_2 & \dots & a_E \\ b_1 & b_2 & \dots & b_E \end{bmatrix}_{2 \times E}$$

که در آن a و b دو بردار به طول E (تعداد یال‌ها) هستند و خروجی یک ماتریس $2 \times E$ خواهد بود که هر ستون آن نمایانگر یک یال با دو گره مبدأ و مقصد است. در کد، این عملیات به صورت torch.stack([src, dst]) پیاده‌سازی شده و خروجی دقیقاً همان ماتریس مورد انتظار PyTorch Geometric برای نمایش لیست یال‌ها است.

```
Data.idx2chemical = {i: chemical_id_to_name[chem_id] for i, chem_id in
enumerate(chemical_ids)}

data.idx2gene = {i: gene_id_to_name[gene_id] for i, gene_id in enumerate(gene_ids)}

data.idx2disease = {i: disease_id_to_name[disease_id] for i, disease_id in
enumerate(disease_ids)}
```

برای استفاده در مراحل بعدی (مانند تفسیر خروجی و نمایش)، سه دیکشنری معکوس (از ایندکس به نام) در خود شیء data ذخیره می‌شود. این کار امکان بازیابی نام دارو، ژن یا بیماری از روی ایندکس گره را فراهم می‌کند.

```
Data_homo = data.to_homogeneous()
```

متد to_homogeneous() گراف ناهمگن را به یک گراف همگن تبدیل می‌کند. در این تبدیل، تمام گره‌ها بدون توجه به نوع در یک مجموعه واحد قرار می‌گیرند و ایندکس‌های گره‌ها به صورت سراسری بازعددهی می‌شوند. همچنین یک تنسور edge_type به گراف همگن اضافه می‌شود که نوع رابطه هر یال را مشخص می‌کند (برای مثال ۰ برای یال دارو-ژن و ۱ برای یال ژن-بیماری). این تنسور برای مدل R-GCN که نیاز به دانستن نوع هر یال دارد ضروری است.

```
Hom_idx2name = {}
for node_type_str, start_idx in [("chemical", 0),
                                 ("gene", data["chemical"].num_nodes),
                                 ("disease", data["chemical"].num_nodes + data["gene"].num_nodes)]:
    n_nodes = data[node_type_str].num_nodes
    for i in range(n_nodes):
        hom_idx = start_idx + i
```

```
        if node_type_str == “chemical”:
            hom_idx2name[hom_idx] = f”{data.idx2chemical[i]} (Chemical)”
        elif node_type_str == “gene”:
            hom_idx2name[hom_idx] = f”{data.idx2gene[i]} (Gene)”
        elif node_type_str == “disease”:
            hom_idx2name[hom_idx] = f”{data.idx2disease[i]} (Disease)”
```

پس از تبدیل به گراف همگن، ایندکس‌های سراسری جدید برای هر گره مشخص می‌شود. این حلقه با توجه به تعداد گره‌های هر نوع و ترتیب قرارگیری آن‌ها (ابتدا داروها، سپس ژن‌ها و در نهایت بیماری‌ها)، یک دیکشنری hom_idx2name می‌سازد که ایندکس سراسری را به نام گره (به همراه نوع) نگاشت می‌کند. این اطلاعات برای مصورسازی و تفسیر نتایج(مانند ماژول CEPS) حیاتی است.

```
Data_homo.hom_idx2name = hom_idx2name
data_homo.offsets = {
    “chemical”: 0,
    “gene”: data[“chemical”].num_nodes,
    “disease”: data[“chemical”].num_nodes + data[“gene”].num_nodes
}
```

دیکشنری نام‌ها و همچنین آفست‌های هر نوع گره (شروع ایندکس سراسری برای هر نوع) به عنوان ویژگی‌های الحاقی به data_homo اضافه می‌شوند. این آفست‌ها در ماژول‌های بعدی (مانند CEPS) برای تفکیک گره‌ها بر اساس نوع استفاده می‌شوند.

```
If data_homo.num_nodes > 0 and data_homo.num_edges > 0:
    print(“Graph successfully created.”)
    print(“Total nodes:”, data_homo.num_nodes)
    print(“Total edges:”, data_homo.num_edges)
    print(“Edge types:”, data_homo.edge_type.unique())
else:
    print(“Graph creation failed.”)
```

در صورت موفقیت‌آمیز بودن ساخت گراف، آمار کلی آن چاپ می‌شود: تعداد کل گره‌ها (۲۶۷۰)، تعداد کل یال‌ها (۳۳۵۲۸) و انواع یال‌های موجود (۰ و ۱). این اعداد نشان می‌دهند که گراف شامل ۲۶۷۰ گره (شامل ۶۴۵ دارو، ۱۹۰۶ ژن و ۱۱۹ بیماری ، با توجه به آفست‌ها) و ۳۳۵۲۸ یال است. دو نوع یال (۰ و ۱) به ترتیب نمایانگر روابط دارو-ژن و ژن-بیماری هستند.

```
Return data_homo
```

تابع، گراف همگن نهایی را بازمی‌گرداند.

```
If __name__ == "__main__":
    data_homo = graph_hetero()
    if data_homo is not None:
        torch.save(data_homo, "hetero_graph.pt")
        print("Graph + full metadata saved successfully")
```

در بخش اصلی برنامه، تابع graph_hetero فراخوانی شده و در صورت موفقیت، گراف به همراه تمام ابرداده‌ها (شامل hom_idx2name و offsets) در فایل hetero_graph.pt ذخیره می‌شود. این فایل در مراحل بعدی (آموزش مدل، استخراج مسیرها و مصورسازی) بارگذاری خواهد شد.

۱.۲.تفسیر خروجی:

خروجی کنسول ماژول hetero_graph.py به صورت زیر است:

```
After Gene merge: (142254, 5)
Before MONDO drop: (146852, 7)
Mapped MONDO count: 106278
Unmapped diseases saved: 932
Children of root: ['0000376', '0000621', '0000637', …]
Cancer set size: 2293
```

Final CEPS shape: (106278, 7)

Unique cancer MONDOs: 4529

Final CEPS shape after filter: (16764, 7)

Unique cancer MONDOs after filter: 134

Sample Disease Names (first 20 unique): […]

Input dataset shape: (16764, 7)

Graph successfully created.

Total nodes: 2670

Total edges: 33528

Edge types: tensor([0, 1])

Graph + full metadata saved successfully

بخش ابتدایی خروجی (تا Sample Disease Names) مربوط به اجرای داخلی تابع merge() است که در ابتدای graph_hetero فراخوانی شده است. سپس ابعاد دیتافریم ورودی (۱۶۷۶۴ ردیف) تأیید می‌شود. پس از ساخت گراف، آمار زیر ارائه می‌گردد:

تعداد کل گره‌ها: ۲۶۷۰

این عدد حاصل جمع تعداد داروها، ژن‌ها و بیماری‌های منحصربه‌فرد در داده‌هاست. با توجه به تعداد ۶۴۵ دارو، ۱۹۰۶ ژن و ۱۱۹ بیماری (که از آفست‌ها قابل استنباط است)، مجموع ۲۶۷۰ به دست می‌آید.

تعداد کل یال‌ها: ۳۳۵۲۸

هر یک از ۱۶۷۶۴ ردیف دیتافریم دو یال ایجاد می‌کند: یکی دارو–ژن و دیگری ژن–بیماری. بنابراین انتظار داریم ۱۶۷۶۴ × ۲ = ۳۳۵۲۸ یال داشته باشیم که با خروجی مطابقت دارد.

انواع یال‌ها: ۰ و ۱

دو نوع رابطه در گراف وجود دارد: نوع ۰ (دارو–ژن) و نوع ۱ (ژن–بیماری). این دو نوع در تنسور edge_type ذخیره می‌شوند و در مدل R-GCN برای تمایز بین روابط به کار می‌روند.

پس از تأیید موفقیت‌آمیز بودن ساخت گراف، فایل hetero_graph.pt در دیسک ذخیره می‌شود. این فایل شامل تمام اطلاعات مورد نیاز برای مراحل بعدی پروژه است: خود گراف (شامل edge_index و edge_type)، دیکشنری‌های نام‌ها (hom_idx2name) و آفست‌های نوع گره‌ها. ذخیره‌سازی یکپارچه این ابرداده‌ها سبب می‌شود در مراحل بعدی (مانند آموزش مدل و تفسیر خروجی) نیازی به بارگذاری مجدد داده‌های خام نباشد و تمام اطلاعات لازم در دسترس باشد.

ماژول hetero_graph.py نقش پل ارتباطی بین داده‌های جدولی و مدل‌سازی گرافی را ایفا می‌کند. با تبدیل داده‌های یکپارچه شده به یک گراف همگن حاوی دو نوع رابطه، زیرساخت لازم برای آموزش یک شبکه عصبی گرافی رابطه‌ای فراهم می‌گردد. نگاشت شناسه‌های اصلی به ایندکس‌های عددی و ذخیره‌سازی ابرداده‌های مرتبط با هر گره، امکان تفسیرپذیری نتایج را در مراحل بعدی (مانند استخراج مسیرهای دارو-ژن-بیماری) میسر می‌سازد. این رویکرد با الهام از معماری Decagon [1] و R-GCN [2]طراحی شده و گامی اساسی در جهت ساخت یک سامانه محاسباتی برای بازتوسعه داروهای سرطان به شمار می‌رود.

۲. ماژول RGCN.py:آموزش مدل گراف عصبی رابطه‌ای

پس از ساخت گراف دانش و ذخیره آن در فایل hetero_graph.pt، گام بعدی آموزش یک مدل شبکه عصبی گرافی است که بتواند ارتباطات جدید میان داروها و بیماری‌ها را پیش‌بینی کند. در معماری Decagon[1]، یک شبکه کانولوشنی گرافی چندلایه با قابلیت مدل‌سازی روابط چندگانه (R-GCN) به همراه یک رمزگشای برداری برای پیش‌بینی لبه‌های برچسب‌دار به کار گرفته شده است. در پروژه حاضر، با توجه به داده‌های موجود (روابط دارو-ژن و ژن-بیماری)، یک مدل رمزگذار-رمزگشا [22] طراحی شده است که در آن رمزگذار یک شبکه R-GCN سه‌لایه و رمزگشا یک مدل DistMult برای محاسبه امتیاز یال‌هاست. این ترکیب امکان بهره‌گیری از ساختار گراف و همچنین تعاملات بین بردارهای بازنمایی گره‌ها را فراهم می‌آورد.

ماژول RGCN.py با هدف آموزش این مدل روی گراف ساخته‌شده طراحی شده است. در این ماژول، ابتدا گراف بارگذاری شده، سپس معماری رمزگذار و رمزگشا تعریف می‌گردد. داده‌ها به مجموعه‌های

[22] encoder–decoder

آمـوزش و اعتبارسـنجی تقسـیم می‌شـوند (بـا در نظـر گـرفتن نـوع رابطـه و جلـوگیری از نشـت اطلاعـات). نمونـه‌های منفـی بـه روش آگـاه از رابطـه [۲۳]مشـابه رویکردهـای اسـتاندارد در تعبیه‌سـازی دانش‌گـراف تولیـد می‌شـوند. در نهایـت مـدل بـا بهینه‌سـاز Adam آمـوزش دیـده و بهتـرین مـدل بـر اسـاس AUC روی مجموعه اعتبارسنجی ذخیره می‌گردد.

۲.۱.کد ماژول :

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import RGCNConv
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
import json
import numpy as np
```

در خطـوط ابتـدایی، کتابخانـه‌های مـورد نیـاز وارد می‌شـوند .torch و زیرمجموعـه‌های آن بـرای سـاخت و آمـوزش شـبکه عصـبی بـه کـار می‌رونـد .RGCNConv از کتابخانـه torch_geometric پیاده‌سـازی لایـه کانولوشن گرافـی رابطـه‌ای (R-GCN) اسـت کـه در مقالـه [۲] معرفـی شـده اسـت. ایـن لایـه قـادر اسـت بـا در نظـر گـرفتن انـواع مختلـف روابـط، بازنمـایی گره‌هـا را به‌روزرسـانی کنـد. از train_test_split بـرای تقسـیم داده‌هـا بـه مجموعـه‌های آمـوزش و اعتبارسـنجی اسـتفاده می‌شـود . roc_auc_score معیـار ارزیـابی AUC را محاسـبه می‌کنـد. کتابخانـه json بـرای ذخیـره معیارهـای آموزشـی و numpy بـرای عملیات عددی به کار می‌رود.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[INFO] Using device: {device}")
```

[۲۳] relation-aware negative sampling

واحد پردازشی مناسب برای محاسبات تعیین می‌شود. در صورت وجود پردازنده گرافیکی [۲۴]با پشتیبانیCUDA ، محاسبات روی آن انجام می‌گیرد که سرعت آموزش را به شدت افزایش می‌دهد؛ در غیر این صورت از پردازنده مرکزی[۲۵]استفاده می‌شود. این انتخاب برای بهینه‌سازی زمان اجرا حیاتی است.

```
graph_path = "hetero_graph.pt"


try:
    data = torch.load(graph_path).to(device)
    print(f"[INFO] Graph loaded successfully: {data.num_nodes} nodes, {data.num_edges}
edges")
    print(f"[INFO] Edge types: {data.edge_type.unique()}")
except FileNotFoundError:
    raise FileNotFoundError(
        f"{graph_path} not found. Run hetero_graph.py first to build and save the graph."
    )


embedding_dim = 256
hidden_dim = 256
out_dim = 64
rel_embedding_dim = 64
lr = 0.001
weight_decay = 1e-3
epochs = 100
weight_positive = 5.0
early_stopping_patience = 20
```

[۲۴] GPU
[۲۵] CPU

```
warmup_epochs = 10
dropout = 0.6

num_relations = int(data.edge_type.max()) + 1
```

گراف از پیش ساخته‌شده بارگذاری شده و به دستگاه محاسباتی منتقل می‌شود. در صورت نبود فایل، خطایی صادر می‌شود که کاربر را به اجرای ماژول hetero_graph.py راهنمایی می‌کند. فراپارامترهای مدل با توجه به ابعاد گراف و نیاز به تعادل بین پیچیدگی و کارایی تنظیم شده‌اند: ابعاد بردارهای تعبیه‌سازی [۲۶]اولیه ۲۵۶، ابعاد لایه‌های پنهان رمزگذار ۲۵۶، ابعاد خروجی رمزگذار ۶۴، و ابعاد بردارهای بازنمایی روابط نیز ۶۴ در نظر گرفته شده است. نرخ یادگیری ۰.۰۰۱، کاهش وزن [27] ۰.۰۰۱، و حداکثر ۱۰۰ دوره آموزشی انتخاب شده است. وزن کلاس مثبت در تابع زیان (۵.۰) برای مقابله با نامتوازنی داده‌ها (تعداد یال‌های مثبت بسیار کمتر از منفی‌های ممکن) به کار رفته است. تعداد دوره‌های warmup ۱۰ تعیین شده که پس از آن تعبیه‌سازی گره‌ها ثابت می‌شوند. نرخ Dropout ۰.۶ برای جلوگیری از بیش‌برازش در نظر گرفته شده است. تعداد انواع روابط نیز از بیشینه برچسب روابط موجود در گراف به‌دست می‌آید.

```
node_emb = nn.Embedding(data.num_nodes, embedding_dim).to(device)
x = node_emb.weight
```

یک لایه تعبیه‌سازی [۲۸]برای گره‌ها تعریف می‌شود که هر گره را به یک بردار با ابعاد embedding_dim نگاشت می‌کند. این بردارها در ابتدا تصادفی مقداردهی می‌شوند و در طول آموزش (تا دوره warmup_epochs) به‌روزرسانی می‌گردند تا ویژگی‌های گره‌ها را بیاموزند. سپس برای استفاده در رمزگذار، به متغیر x اختصاص می‌یابند.

---

[۲۶] Embedding

[27] weight decay

[۲۸] Embedding

```
class RGCNEncoder(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim, num_relations, dropout=0.6):
        super().__init__()
        self.conv1 = RGCNConv(in_dim, hidden_dim, num_relations)
        self.conv2 = RGCNConv(hidden_dim, hidden_dim, num_relations)
        self.conv3 = RGCNConv(hidden_dim, out_dim, num_relations)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, edge_index, edge_type):
        x = F.relu(self.conv1(x, edge_index, edge_type))
        x = self.dropout(x)
        x = F.relu(self.conv2(x, edge_index, edge_type))
        x = self.dropout(x)
        x = self.conv3(x, edge_index, edge_type)
        return x
```

کلاس RGCNEncoder یک رمزگذار گرافی سه‌لایه است. هر لایه از نوع RGCNConv استفاده می‌کند که در مقاله [۲] معرفی شده است. عملکرد این لایه به صورت زیر است:

$$h_i^{(l+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\frac{1}{c_{i,r}}W_r^{(l)}h_j^{(l)} + W_0^{(l)}h_i^{(l)}\right)$$

که در $\mathcal{N}_i^r$ آن مجموعه همسایگان گره i با رابطه r است، $W_r^{(l)}$ ماتریس وزن مخصوص رابطه r در لایه l، و $c_{i,r}$ یک ثابت نرمال‌ساز (معمولاً تعداد همسایگان) می‌باشد. این لایه‌ها به طور موازی برای همه گره‌ها محاسبه می‌شوند و امکان انتشار اطلاعات در طول گراف با در نظر گرفتن انواع روابط را فراهم می‌کنند.

در این معماری، سه لایه پشت سر هم قرار گرفته‌اند: لایه اول ابعاد ورودی (۲۵۶) را به ۲۵۶ نگاشت می‌کند، لایه دوم همان ابعاد را حفظ می‌کند و لایه سوم خروجی با ابعاد ۶۴ تولید می‌کند. پس از هر لایه به جز آخرین، تابع ReLU و dropout با نرخ ۰.۶ اعمال می‌شود. خروجی نهایی، بازنمایی فشرده‌شده هر گره است که اطلاعات ساختاری گراف را در خود دارد.

```
class DistMultDecoder(nn.Module):
    def __init__(self, num_relations, node_emb_dim, rel_emb_dim):
        super().__init__()
        self.relation_emb = nn.Embedding(num_relations, rel_emb_dim)
        self.proj = nn.Linear(node_emb_dim, rel_emb_dim)

    def forward(self, z, edge_index, edge_type):
        head, tail = edge_index[0], edge_index[1]
        h = self.proj(z[head])
        t = self.proj(z[tail])
        r = self.relation_emb(edge_type)
        return (h * r * t).sum(dim=-1)
```

کلاس DistMultDecoder رمزگشایی است که بر اساس مدل DistMult عمل می‌کند. در این مدل، امتیاز یک یال (سر، رابطه، دم) به صورت ضرب داخلی (ضرب نقطه‌ای) بردارهای بازنمایی گره سر، بازنمایی رابطه و بازنمایی گره دم محاسبه می‌شود:

$$f(s, r, o) = \mathbf{e}_s^T \mathbf{R}_r \mathbf{e}_o$$

که در آن Rr یک ماتریس قطری است. در پیاده‌سازی حاضر، ابتدا بردارهای بازنمایی گره‌ها (با ابعاد (out_dim = 64)توسط یک لایه خطی [۲۹]به ابعاد rel_emb_dim = 64 نگاشت می‌شوند. سپس بردار بازنمایی رابطه (r) از لایه تعبیه‌سازی مخصوص روابط استخراج می‌گردد. در نهایت حاصل ضرب عنصری [۳۰]سه بردار محاسبه و جمع روی بعد آخر (که معادل ضرب داخلی است) به عنوان امتیاز برگردانده می‌شود. این امتیاز سپس توسط تابع سیگموید به احتمال تبدیل خواهد شد.

[۲۹] self.proj

[۳۰] Hadamard product

```
encoder = RGCNEncoder(embedding_dim, hidden_dim, out_dim, num_relations,
dropout).to(device)

decoder = DistMultDecoder(num_relations, out_dim, rel_embedding_dim).to(device)
```

نمونه‌هایی از رمزگذار و رمزگشا ساخته شده و به دستگاه محاسباتی منتقل می‌شوند.

```
all_edges = data.edge_index.cpu().numpy().T

all_types = data.edge_type.cpu().numpy()


train_edges_list = []

val_edges_list = []

train_types_list = []

val_types_list = []


for rel in range(num_relations):

    mask = all_types == rel

    rel_edges = all_edges[mask]

    if len(rel_edges) == 0:

        continue

    train, val = train_test_split(rel_edges, test_size=0.2, random_state=42)

    train_edges_list.append(train)

    val_edges_list.append(val)

    train_types_list.extend([rel] * len(train))

    val_types_list.extend([rel] * len(val))


train_edges = torch.tensor(np.concatenate(train_edges_list, axis=0),
dtype=torch.long).t().to(device)

val_edges = torch.tensor(np.concatenate(val_edges_list, axis=0),
dtype=torch.long).t().to(device)
```

```
train_types = torch.tensor(train_types_list, dtype=torch.long).to(device)
val_types = torch.tensor(val_types_list, dtype=torch.long).to(device)

edge_index_train = train_edges
edge_type_train = train_types

print(f"[INFO] Train edges: {train_edges.size(1)}")
print(f"[INFO] Val edges: {val_edges.size(1)}")
```

برای ارزیابی واقع‌بینانه، باید از نشت اطلاعات بین مجموعه‌های آموزش و اعتبارسنجی جلوگیری شود. به همین دلیل، یال‌ها بر اساس نوع رابطه تفکیک شده و هر نوع به طور جداگانه با نسبت ۸۰-۲۰ تقسیم می‌شود. این کار تضمین می‌کند که هر دو نوع رابطه در هر دو مجموعه حضور داشته باشند و توزیع انواع در آموزش و اعتبارسنجی مشابه باشد.

ابتدا تمام یال‌ها و انواع آن‌ها به آرایه‌های numpy تبدیل می‌شوند. سپس برای هر نوع رابطه، یال‌های مربوطه استخراج و با train_test_split تقسیم می‌گردند. یال‌های آموزش و اعتبارسنجی هر نوع در لیست‌های جداگانه جمع‌آوری و نهایتاً با np.concatenate ادغام می‌شوند. سپس به تنسور تبدیل شده و به دستگاه محاسباتی منتقل می‌شوند. تعداد یال‌های آموزش (۲۶۸۲۲) و اعتبارسنجی (۶۷۰۶) در خروجی چاپ می‌شود.

```
def relation_aware_negative_sampling(pos_edges, pos_types, num_nodes,
num_neg_per_pos=3):
    pos_edges = pos_edges.t()
    E = pos_edges.size(0)
    total_neg = E * num_neg_per_pos

    neg_edges = torch.zeros(total_neg, 2, dtype=torch.long, device=pos_edges.device)
    neg_types = torch.zeros(total_neg, dtype=torch.long, device=pos_types.device)
```

```
    for i in range(num_neg_per_pos):
        start = i * E
        end = (i + 1) * E
        neg_edges[start:end] = pos_edges.clone()
        neg_types[start:end] = pos_types

        corrupt_head = torch.randint(0, 2, (E,), device=pos_edges.device).bool()
        random_nodes = torch.randint(0, num_nodes, (E,), device=pos_edges.device)
        neg_edges[start:end, 0] = torch.where(corrupt_head, random_nodes, pos_edges[:, 0])
        neg_edges[start:end, 1] = torch.where(~corrupt_head, random_nodes, pos_edges[:, 1])

    return neg_edges.t().contiguous(), neg_types
```

این تابع نمونه‌گیری منفی آگاه از رابطه را پیاده‌سازی می‌کند. در مدل‌های تعبیه‌سازی دانش‌گراف، معمولاً برای هر یال مثبت، چند یال منفی با خراب‌کردن [۳۱]یکی از دو سر یال (جایگزینی با یک گره تصادفی) ساخته می‌شود. در اینجا برای هر یال مثبت، num_neg_per_pos = 3یال منفی تولید می‌شود. نوع رابطه در یال‌های منفی دقیقاً مشابه یال مثبت باقی می‌ماند (از این رو «آگاه از رابطه»). این کار باعث می‌شود مدل یاد بگیرد که برای یک رابطه مشخص، کدام جفت گره‌ها معتبر هستند و کدام نه.

حلقه به تعداد نمونه‌های منفی مورد نظر می‌چرخد. در هر تکرار، ابتدا یال‌های مثبت کپی می‌شوند. سپس به طور تصادفی تصمیم گرفته می‌شود که آیا سر [۳۲]خراب شود یا دم [33].یک گره تصادفی انتخاب و جایگزین قسمت مورد نظر می‌شود. در نهایت، یال‌های منفی به صورت یک بردار با ابعاد (E * num_neg_per_pos) × 2برگردانده می‌شوند.

```
optimizer = torch.optim.Adam(
    list(encoder.parameters()) + list(decoder.parameters()) + list(node_emb.parameters()),
```

[۳۱] corrupting
[۳۲] head
[33] tail

```
    lr=lr, weight_decay=weight_decay
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5,
patience=10, verbose=True)
criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(weight_positive, device=device))

metrics = {"train_loss": [], "val_loss": [], "val_auc": []}
best_val_auc = 0
patience_counter = 0
print("\n[INFO] Starting training with relation-aware negative sampling...\n")

for epoch in range(1, epochs + 1):
    encoder.train()
    decoder.train()
    optimizer.zero_grad()

    z = encoder(x, edge_index_train, edge_type_train)

    neg_edges, neg_types = relation_aware_negative_sampling(
        train_edges, train_types, data.num_nodes, num_neg_per_pos=3
    )

    pos_scores = decoder(z, train_edges, train_types)
    neg_scores = decoder(z, neg_edges, neg_types)

    y = torch.cat([
        torch.ones(pos_scores.size(0), device=device),
        torch.zeros(neg_scores.size(0), device=device)
    ])
```

```
    loss = criterion(torch.cat([pos_scores, neg_scores]), y)

    loss.backward()
    torch.nn.utils.clip_grad_norm_(
        list(encoder.parameters()) + list(decoder.parameters()) + list(node_emb.parameters()),
        max_norm=5.0
    )
    optimizer.step()

    if epoch == warmup_epochs:
        node_emb.requires_grad_(False)
        optimizer = torch.optim.Adam(
            list(encoder.parameters()) + list(decoder.parameters()),
            lr=lr, weight_decay=weight_decay
        )
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, mode='max', factor=0.5, patience=10, verbose=True
        )
        print(f"[INFO] Node embeddings frozen after epoch {warmup_epochs}. Optimizer
recreated.")

    encoder.eval()
    decoder.eval()
    with torch.no_grad():
        neg_val_edges, neg_val_types = relation_aware_negative_sampling(
            val_edges, val_types, data.num_nodes, num_neg_per_pos=3
        )

        val_pos_scores = decoder(z, val_edges, val_types)
```

```
        val_neg_scores = decoder(z, neg_val_edges, neg_val_types)

        val_y = torch.cat([
            torch.ones(val_pos_scores.size(0)),
            torch.zeros(val_neg_scores.size(0))
        ]).cpu().numpy()
        val_scores = torch.sigmoid(torch.cat([val_pos_scores, val_neg_scores])).cpu().numpy()
        val_loss = criterion(
            torch.cat([val_pos_scores, val_neg_scores]),
            torch.tensor(val_y, device=device)
        ).item()
        val_auc = roc_auc_score(val_y, val_scores)

    metrics["train_loss"].append(loss.item())
    metrics["val_loss"].append(val_loss)
    metrics["val_auc"].append(val_auc)

    scheduler.step(val_auc)

    print(f"Epoch {epoch:03d} | Train Loss: {loss.item():.4f} | Val Loss: {val_loss:.4f} | Val
AUC: {val_auc:.4f}")
    if val_auc > best_val_auc:
        best_val_auc = val_auc
        patience_counter = 0

        torch.save({
            "encoder_state_dict": encoder.state_dict(),
            "decoder_state_dict": decoder.state_dict(),
            "node_emb_state_dict": node_emb.state_dict(),
```

```
            "embedding_dim": embedding_dim,
            "hidden_dim": hidden_dim,
            "out_dim": out_dim,
            "rel_embedding_dim": rel_embedding_dim,
            "num_relations": num_relations,
            "decoder_type": "DistMult"
        }, "rgcn_distmult_best.pt")

        print("[INFO] Best model saved.")
    else:
        patience_counter += 1
        if patience_counter >= early_stopping_patience:
            print(f"[INFO] Early stopping at epoch {epoch}")
            break
with open("rgcn_metrics.json", "w") as f:
    json.dump(metrics, f, indent=2)

print("\nTraining complete.")
print(f"Best Val AUC: {best_val_auc:.4f}")
```

این بخش هسته اصلی آموزش مدل است. بهینه‌ساز Adam با نرخ یادگیری ۰.۰۰۱ و کاهش وزن[34] ۰.۰۰۱ برای به‌روزرسانی پارامترهای رمزگذار، رمزگشا و لایه تعبیه‌سازی تعریف می‌شود. یک برنامه‌ریز نرخ یادگیری[۳۵]نیز تنظیم می‌گردد که در صورت عدم بهبود AUC اعتبارسنجی پس از ۱۰ دوره، نرخ یادگیری را به نصف کاهش می‌دهد. تابع زیان، آنتروپی متقاطع دودویی[۳۶]با Logits انتخاب شده و وزن کلاس مثبت (۵.۰) به آن اعمال می‌شود تا مشکل نامتوازنی داده‌ها (تعداد یال‌های مثبت بسیار کمتر از منفی‌های تولیدشده) جبران گردد.

[34] weight decay
[۳۵] ReduceLROnPlateau
[۳۶] Binary Cross-Entropy with Logits

در هر دوره آموزشی:

- رمزگذار و رمزگشا در حالت آموزش قرار می‌گیرند و گرادیان‌ها بازنشانی می‌شوند.
- بازنمایی گره‌ها (z) با عبور بردارهای تعبیه‌سازی اولیه از رمزگذار محاسبه می‌گردد.
- نمونه‌های منفی برای مجموعه آموزش با استفاده از تابع relation_aware_negative_samplingتولید می‌شوند.
- امتیازهای مثبت و منفی از رمزگشا دریافت شده و در یک تنسور ترکیب می‌گردند.
- تابع زیان محاسبه، گرادیان‌ها پس‌گسترش یافته و پارامترها به‌روزرسانی می‌شوند.
- برای جلوگیری از انفجار گرادیان [۳۷]، clip_grad_norm برش می‌خورد[۳۸].

پس از ۱۰ دوره مقدارwarmup_epochs ، لایه تعبیه‌سازی گره‌ها ثابت می‌شود (آموزش آن متوقف می‌گردد) و بهینه‌ساز مجدداً تعریف می‌شود تا تنها پارامترهای رمزگذار و رمزگشا به‌روزرسانی شوند. این کار سبب می‌شود مدل بر یادگیری بازنمایی‌های ساختاری از گراف متمرکز شود و از تغییرات ناگهانی در تعبیه‌سازی‌های اولیه جلوگیری کند.

پس از هر دوره، مدل در حالت ارزیابی قرار گرفته و عملکرد آن روی مجموعه اعتبارسنجی سنجیده می‌شود. نمونه‌های منفی جدید برای اعتبارسنجی نیز تولید می‌گردند تا ارزیابی منصفانه باشد. معیار[۳۹] AUC به‌عنوان شاخص اصلی انتخاب بهترین مدل محاسبه می‌شود. اگر AUC اعتبارسنجی بهبود یابد، مدل فعلی (شامل وزن‌های رمزگذار، رمزگشا و لایه تعبیه‌سازی) در فایل rgcn_distmult_best.ptذخیره می‌گردد. در غیر این صورت، شمارنده توقف زودهنگام افزایش می‌یابد و در صورت عدم بهبود پس از ۲۰ دوره، آموزش متوقف می‌شود.

در پایان، معیارهای آموزشی شامل(تابع زیان و AUC در هر دوره) در یک فایل JSON ذخیره شده و بهترین AUC اعتبارسنجی چاپ می‌گردد.

---

[۳۷] Gradient Explosion

[۳۸] clip_grad_norm

[۳۹] Area Under the ROC Curve

۲.۲.تفسیر خروجی:

خروجی‌های کنسول در طول آموزش، نمایانگر روند یادگیری مدل هستند. کاهش تدریجی تابع زیان روی مجموعه آموزش نشان‌دهنده یادگیری صحیح مدل است. افزایش AUC اعتبارسنجی بیانگر توانایی مدل در تشخیص یال‌های درست از نادرست (یال‌های مثبت از منفی) می‌باشد. مقدار نهایی AUCکه معمولاً در بازه ۰.۸ تا ۰.۹ قرار می‌گیرد، عملکرد قابل قبول مدل را تأیید می‌کند و نشان می‌دهد که بازنمایی‌های آموخته‌شده توسط رمزگذارR-GCN ، اطلاعات مفیدی برای پیش‌بینی روابط دارو–ژن و ژن–بیماری فراهم آورده است.

پس از پایان آموزش، بهترین مدل ذخیره‌شده در مراحل بعدی(مانند استخراج مسیرهای توضیحی با GNN Explainer)مورد استفاده قرار خواهد گرفت. همچنین فایل معیارهای آموزشی امکان تحلیل دقیق‌تر روند یادگیری و تشخیص مشکلات احتمالی مانند بیش‌برازش [۴۰]را فراهم می‌آورد.

۳ .ماژول GNN_Explainer.py: استخراج مسیرهای توضیحی

پس از آموزش مدل R-GCN و ذخیره بازنمایی‌های گره‌ها، گام بعدی ارائه توضیح برای پیش‌بینی‌های مدل است. در بسیاری از کاربردهای زیست‌پزشکی، صرف پیش‌بینی یک ارتباط جدید کافی نیست؛ بلکه پژوهشگران نیاز دارند بدانند کدام مسیرهای زیستی (توالی دارو ← ژن ← بیماری) منجر به این پیش‌بینی شده‌اند. به این ترتیب می‌توان فرضیه‌های قابل آزمایش تجربی تولید کرد. ماژول GNN_Explainer.py با الهام از روش‌های توضیح‌پذیری برای شبکه‌های گرافی، یک رویکرد پس‌ازآموزش [41] برای شناسایی یال‌ها و گره‌های مهم در یک پیش‌بینی خاص ارائه می‌دهد.

در این ماژول، توابعی تعریف می‌شوند که با دریافت بازنمایی گره‌ها (خروجی رمزگذار) و یک جفت دارو-بیماری مشخص، اهمیت هر یال را به‌گونه‌ای یاد می‌گیرند که امتیاز آن جفت در مدل اصلی حفظ شود. سپس با تجمیع اهمیت یال‌ها، گره‌های مهم استخراج شده و مسیرهای دارو-ژن-بیماری که از این گره‌ها عبور می‌کنند، شمارش می‌گردند. توجه به این نکته ضروری است که این ماژول به‌تنهایی اجرا نمی‌شود و توابع آن در main.py فراخوانی می‌گردند.

---

[۴۰] Overfitting

[41] post-hoc

۳.۱ .کد ماژول:

```
import torch
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[INFO] GNN Explainer running on: {device}")
```

کتابخانه‌های پایه‌ای PyTorch برای کار با بردارها و تعریف پارامترهای قابل یادگیری وارد می‌شوند. دستگاه محاسباتی (GPU یا CPU)تعیین شده و یک پیام اطلاع‌رسانی چاپ می‌گردد.

```
def explain_link_edge_aware(
    z,
    edge_index,
    src,
    dst,
    link_score,
    epochs=300,
    lr=0.1,
    lam_sparse=0.01,
    lam_entropy=0.01,
    verbose=True
):
```

این تابع هسته اصلی توضیح‌دهنده است. ورودی‌های آن عبارتند از:

- z:بازنمایی نهایی گره‌ها(خروجی رمزگذار R-GCN) با ابعاد (تعداد گره‌ها × ابعاد بازنمایی)
- edge_index:لیست یال‌های گراف (با ابعاد ۲ × تعداد یال‌ها)
- src:گره مبدأ (دارو) مورد نظر
- dst:گره مقصد (بیماری) مورد نظر

- link_score :تابع امتیازدهی به یال که در ماژول CEPS تعریف شده (برای محاسبه امتیاز شباهت کسینوسی بین دو گره)
- epochs:تعداد دوره‌های بهینه‌سازی (پیش‌فرض ۳۰۰)
- lr:نرخ یادگیری برای به‌روزرسانی پارامتر ماسک یال‌ها (پیش‌فرض ۰.۱)
- lam_sparse :ضریب جریمه برای پراکندگی [۴۲]ماسک – تشویق به فعال بودن تعداد کمی یال
- lam_entropy : ضریب جریمه آنتروپی – جلوگیری از قطبی شدن بیش از حد ماسک
- verbose :نمایش جزئیات در طول بهینه‌سازی

```
device = z.device
num_edges = edge_index.size(1)

edge_mask_param = nn.Parameter(
    torch.randn(num_edges, device=device)
)

optimizer = torch.optim.Adam([edge_mask_param], lr=lr)

with torch.no_grad():
    base_score = link_score(src, dst)
```

ابتدا دستگاه محاسباتی از روی بازنمایی گره‌ها گرفته می‌شود. تعداد کل یال‌های گراف مشخص می‌گردد. یک پارامتر قابل یادگیری به نام edge_mask_param با ابعاد تعداد یال‌ها و مقادیر اولیه تصادفی از توزیع نرمال استاندارد تعریف می‌شود. این پارامتر پس از عبور از تابع سیگموید، ماسک نهایی یال‌ها (اهمیت هر یال) را می‌سازد. بهینه‌ساز Adam برای به‌روزرسانی این پارامتر با نرخ

[۴۲] sparsity

یادگیری داده شده تنظیم می‌شود. سپس امتیاز پایه [۴۳]برای جفت[۴۴]با استفاده از تابع link_score و بدون تأثیر ماسک محاسبه و ذخیره می‌گردد. این امتیاز هدفی است که تلاش می‌کنیم با ماسک‌کردن بخشی از گراف همچنان حفظ شود.

```
for epoch in range(epochs):
    mask = torch.sigmoid(edge_mask_param)
    node_weight = torch.zeros(z.size(0), device=device)
    node_weight.index_add_(0, edge_index[0], mask)
    node_weight.index_add_(0, edge_index[1], mask)

    node_weight = node_weight / (node_weight.max() + 1e-8)
    z_masked = z * node_weight.unsqueeze(1)
```

در هر دوره، ماسک یال‌ها با اعمال تابع سیگموید روی پارامتر قابل یادگیری محاسبه می‌شود (مقادیر بین ۰ و ۱). سپس اهمیت هر گره با جمع اهمیت یال‌های متصل به آن محاسبه می‌گردد. برای این کار از عملیات برداری index_add_استفاده می‌شود که بسیار سریع‌تر از حلقه صریح است. پس از نرمال‌سازی وزن گره‌ها (تقسیم بر بیشینه مقدار)، بازنمایی گره‌ها در z_masked با ضرب کردن Zدر وزن متناظر هر گره به‌صورت عنصری [۴۵]ماسک می‌شود. به این ترتیب گره‌های مهم‌تر سهم بیشتری در بازنمایی خواهند داشت.

```
    pred = link_score(src, dst, z_override=z_masked, return_tensor=True)

    loss_pred = (pred - base_score) ** 2
    loss_sparse = mask.mean()
    loss_entropy = (-mask * torch.log(mask + 1e-8)).mean()

    loss = loss_pred + lam_sparse * loss_sparse + lam_entropy * loss_entropy
```

[۴۳] base_score
[۴۴] src, dst
[۴۵] broadcast

امتیـاز جدیـد بـرای جفـت بـا اسـتفاده از بازنمـایی ماسک‌شـده محاسـبه می‌شـود. تـابع زیـان از سـه جـزء تشکیل شده است:

- loss_pred : مربع خطـای امتیـاز جدیـد نسـبت بـه امتیـاز پایـه – تـلاش می‌کنـد پیش‌بینـی پـس از ماسک‌کردن تغییر چندانی نکند.
- loss_sparse : میـانگین مقـادیر ماسـک – بـا کـم کـردن ایـن عبـارت (چـون بـا ضـریب مثبـت در زیان ظاهر می‌شود)، مـدل تشـویق می‌شـود ماسـک را پراکنـده (یعنـی تعـداد کمـی یـال بـا اهمیـت بالا) نگه دارد.
- loss_entropy : آنتروپـی ماسـک – حـداقل شـدن آنتروپـی معـادل قطبی‌تـر شـدن ماسـک اسـت (مقادیر نزدیک به ۰ یا ۱)، که از مبهم بودن اهمیت یال‌ها جلوگیری می‌کند.

```
optimizer.zero_grad()
loss.backward()
optimizer.step()

if verbose and epoch % 50 == 0:
   print(f"[Epoch {epoch:03d}] loss={loss.item():.4f} pred={pred.item():.4f}")
```

گرادیان‌هـا محاسـبه شـده و پـارامتر ماسـک به‌روزرسـانی می‌شـود. هـر ۵۰ دوره، مقـدار زیـان و امتیـاز پیش‌بینی چاپ می‌گردد.

```
return torch.sigmoid(edge_mask_param).detach()
```

پـس از پایـان بهینه‌سـازی، ماسـک نهـایی یال‌هـا (بـا اعمـال سـیگموید روی پـارامتر نهـایی) بازگردانـده می‌شـود. تـابع detachباعـث می‌شـود ایـن تنسـور از گـراف محاسـباتی جـدا شـده و بـه‌عنوان یـک تنسـور معمولی قابل استفاده باشد.

```
def important_nodes_from_edge_mask(edge_mask, edge_index, ratio=0.4):
   num_nodes = int(edge_index.max()) + 1
```

```
    node_score = torch.zeros(num_nodes, device=edge_mask.device)

    node_score.index_add_(0, edge_index[0], edge_mask)
    node_score.index_add_(0, edge_index[1], edge_mask)

    k = max(1, int(num_nodes * ratio))
    return set(node_score.topk(k).indices.tolist())
```

این تابع ماسک اهمیت یال‌ها را دریافت کرده و اهمیت هر گره را با جمع اهمیت یال‌های متصل به آن محاسبه می‌کند (باز هم با استفاده از index_add_ برداری). سپس با نسبت داده شده (پیش‌فرض ۰.۴)، تعداد k گره با بیشترین اهمیت را انتخاب کرده و مجموعه‌ای از ایندکس‌های آن‌ها را بازمی‌گرداند. این گره‌ها به‌عنوان «گره‌های مهم» در توضیح مشارکت دارند.

```
def enumerate_ceps_paths_masked(
    important_nodes,
    adj,
    chemical_nodes,
    gene_nodes,
    disease_nodes,
    link_score
):
    paths = []

    for c in chemical_nodes:
        if c not in important_nodes:
            continue

        for g in gene_nodes:
            if g not in important_nodes or g not in adj[c]:
```

```
            continue

        for d in disease_nodes:
            if d not in important_nodes or d not in adj[g]:
                continue

            score = link_score(c, g) + link_score(g, d)
            paths.append((score, [c, g, d]))

    paths.sort(reverse=True, key=lambda x: x[0])
    return paths
```

این تابع مسیرهای دارو–ژن–بیماری را تنها برای گره‌هایی که در مجموعه important_nodes قرار دارند شمارش می‌کند. بدین منظور، روی همه داروها، ژن‌ها و بیماری‌ها پیمایش انجام می‌شود و تنها زمانی یک مسیر ثبت می‌گردد که هر سه گره در مجموعه گره‌های مهم باشند و همچنین یال دارو–ژن (با چک کردن adj[c]) و یال ژن–بیماری (با adj[g])وجود داشته باشد. امتیاز هر مسیر حاصل جمع امتیاز دو یال (با تابع link_score) است. در پایان مسیرها بر اساس امتیاز نزولی مرتب می‌شوند.

```
def explain_and_enumerate_paths(
    z,
    edge_index,
    src,
    dst,
    adj,
    chemical_nodes,
    gene_nodes,
    disease_nodes,
    link_score,
```

```
    node_ratio=0.4,
    epochs=300
):
    print("[INFO] Running GNN Explainer (edge-aware)...")

    edge_mask = explain_link_edge_aware(
        z=z,
        edge_index=edge_index,
        src=src,
        dst=dst,
        link_score=link_score,
        epochs=epochs
    )

    important_nodes = important_nodes_from_edge_mask(
        edge_mask, edge_index, ratio=node_ratio
    )

    print(f"[INFO] Important nodes selected: {len(important_nodes)}")

    print("[INFO] Enumerating CEPS-style paths...")
    paths = enumerate_ceps_paths_masked(
        important_nodes,
        adj,
        chemical_nodes,
        gene_nodes,
        disease_nodes,
        link_score
    )
```

```
print(f"[INFO] Total paths found: {len(paths)}")

return {
    "edge_mask": edge_mask,
    "important_nodes": important_nodes,
    "paths": paths
}
```

این تابع کل فرایند توضیح‌دهی را یکپارچه می‌کند:

۱. اجرای explain_link_edge_aware برای یادگیری ماسک یال‌ها.

۲. استخراج گره‌های مهم با important_nodes_from_edge_mask .

۳. شمارش مسیرهای CEPS با enumerate_ceps_paths_masked .

۴. بازگرداندن یک دیکشنری شامل ماسک یال‌ها، مجموعه گره‌های مهم و لیست مسیرها.

این تابع در main.py برای یک جفت دارو–بیماری خاص فراخوانی خواهد شد.

۳.۲. تفسیر خروجی:

ماژول GNN_Explainer.py به‌تنهایی اجرا نمی‌شود و تنها توابع کمکی برای توضیح‌دهی را فراهم می‌کند. خروجی آن زمانی قابل مشاهده است که در main.py فراخوانی شود. در آنجا پیام‌هایی مانند "[INFO] Running GNN Explainer..."، تعداد گره‌های مهم انتخاب‌شده و تعداد مسیرهای یافت‌شده در کنسول نمایش داده می‌شود.

۴ . ماژول CEPS.py: شمارش و مصورسازی مسیرها

ماژول CEPS.py وظیفه بارگذاری مدل آموزش‌دیده، محاسبه بازنمایی نهایی گره‌ها، شمارش تمام مسیرهای دارو-ژن-بیماری (بدون اعمال ماسک) و همچنین مصورسازی تعاملی مسیرها را بر عهده

دارد. ایـن مـاژول هـم به‌صـورت مسـتقل (بـرای اجـرای پایـه) و هـم بـه‌عنوان سـرویس‌دهنده توابـع کمکـی برای main.py قابل استفاده است.

۴.۱. کد ماژول:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import RGCNConv
import networkx as nx
import plotly.graph_objects as go
import webbrowser
```

کتابخانـــه‌های مـــورد نیـــاز وارد می‌شـــوند:PyTorch بـــرای عملیـــات تنســـوری، PyTorch Geometricبـــرای لایـــهRGCN ، networkxبـــرای ســـاخت گـــراف ســـاده، plotly بـــرای مصورســـازی تعـــاملی، webbrowserبـــرای باز کردن خودکار خروجی در مرورگر.

```
data = None
z = None
edge_index = None
edge_type = None
adj = None

chemical_nodes = None
gene_nodes = None
disease_nodes = None
offsets = None

all_paths = None
paths_by_pair = None
```

متغیرهـای سراسـری کـه در فراینـد مقـداردهی اولیـه پـر می‌شـوند. ایـن کـار باعـث می‌شـود پـس از یک‌بـار فراخوانی ()initialize، سایر توابع به داده‌ها دسترسی داشته باشند.

```
def initialize():
    global data, z, edge_index, edge_type, adj
    global chemical_nodes, gene_nodes, disease_nodes
    global offsets

    if z is not None:
        return

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[INFO] Initializing CEPS on: {device}")

    data = torch.load("hetero_graph.pt", map_location=device, weights_only=False).to(device)

    num_nodes = data.num_nodes
    edge_index = data.edge_index
    edge_type = data.edge_type
    offsets = data.offsets

    chemical_nodes = list(range(offsets["chemical"], offsets["gene"]))
    gene_nodes     = list(range(offsets["gene"], offsets["disease"]))
    disease_nodes  = list(range(offsets["disease"], num_nodes))
```

این تابع گراف را از فایل بارگذاری کرده، متغیرهای سراسری را مقداردهی می‌کند و ایندکس گره‌ها را بر اساس نوع جدا می‌کند. آفست‌ها که در hetero_graph.py ذخیره شده بودند، مشخص می‌کنند که هر دسته از گره‌ها از

کجا شـروع می‌شـوند. اگر پیش‌تر مقداردهی انجام شـده باشـد (بررسـی z is not None)، تابع بدون عملیات بازمی‌گردد.

```
class RGCNEncoder(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim, num_relations, dropout=0.6):
        super().__init__()
        self.conv1 = RGCNConv(in_dim, hidden_dim, num_relations)
        self.conv2 = RGCNConv(hidden_dim, hidden_dim, num_relations)
        self.conv3 = RGCNConv(hidden_dim, out_dim, num_relations)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, edge_index, edge_type):
        x = F.relu(self.conv1(x, edge_index, edge_type))
        x = self.dropout(x)
        x = F.relu(self.conv2(x, edge_index, edge_type))
        x = self.dropout(x)
        x = self.conv3(x, edge_index, edge_type)
        return x
```

معماری رمزگـذار دقیقـاً مشـابه آنچـه در RGCN.py تعریـف شـده بـود، در اینجـا نیـز بـازتعریف می‌شـود تـا بتوان وزن‌های ذخیره‌شده را بارگذاری کرد.

```
checkpoint = torch.load("rgcn_distmult_best.pt", map_location=device,
weights_only=False)

embedding_dim = checkpoint["embedding_dim"]
hidden_dim = checkpoint["hidden_dim"]
out_dim = checkpoint["out_dim"]
num_relations = checkpoint["num_relations"]

node_emb = nn.Embedding(num_nodes, embedding_dim).to(device)
```

```
node_emb.load_state_dict(checkpoint["node_emb_state_dict"])

encoder = RGCNEncoder(embedding_dim, hidden_dim, out_dim, num_relations).to(device)
encoder.load_state_dict(checkpoint["encoder_state_dict"])
encoder.eval()

with torch.no_grad():
    z = encoder(node_emb.weight, edge_index, edge_type).detach()
```

مدل آموزش‌دیده (شامل وزن‌های رمزگذار و لایه تعبیه‌سازی) بارگذاری شده و در حالت ارزیابی قرار می‌گیرد. سپس بازنمایی نهایی گره‌ها (z) با عبور از رمزگذار محاسبه و ذخیره می‌شود.

```
adj = {n: set() for n in range(num_nodes)}
for u, v in edge_index.t().tolist():
    adj[u].add(v)
    adj[v].add(u)

print("[INFO] CEPS initialized ✅")
```

یک لیست مجاورت[۴۶] از روی یال‌های گراف ساخته می‌شود تا در شمارش مسیرها بتوان به‌سرعت وجود یال بین دو گره را بررسی کرد.

```
def link_score(u, v, z_override=None, return_tensor=False):
    emb = z_override if z_override is not None else z

    sim = torch.cosine_similarity(
        emb[u].unsqueeze(0),
        emb[v].unsqueeze(0)
```

[۴۶] adjacency list

```
)
score = (sim + 1) / 2

if return_tensor:
    return score.squeeze()
else:
    return score.item()
```

این تابع امتیاز شباهت بین دو گره را بر اساس شباهت کسینوسی بردارهای بازنمایی آن‌ها محاسبه می‌کند. مقدار شباهت کسینوسی بین ۱- و ۱ است که با تبدیل (sim+1)/2 به بازه [۰,۱] نگاشته می‌شود. اگر z_override داده شود (مثلاً در فرایند توضیح‌دهی)، از آن به جای z استفاده می‌کند. با استفاده از return_tensor مشخص می‌شود که خروجی به‌صورت تنسور خام برگردد یا عدد اعشاری پایتون.

```
paths_by_pair = {}
for global_rank, (score, path) in enumerate(all_paths):
    c, g, d = path
    pair = (c, d)
    paths_by_pair.setdefault(pair, []).append((score, path, global_rank))

for pair in paths_by_pair:
    paths_by_pair[pair].sort(reverse=True, key=lambda x: x[0])
    paths_by_pair[pair] = [
        (score, path, global_rank, local_rank)
        for local_rank, (score, path, global_rank)
        in enumerate(paths_by_pair[pair])
    ]
print(f"[INFO] Total pairs: {len(paths_by_pair)}")

visualize_paths(all_paths, paths_by_pair)
```

ساختار paths_by_pair برای هر جفت (دارو، بیماری) لیست مسیرهای مربوطه را ذخیره می‌کند. این لیست برای هر جفت به‌صورت محلی مرتب و با اندیس محلی [۴۷]تکمیل می‌شود. سپس تابع visualize_pathsفراخوانی می‌شود.

```
def visualize_paths(all_paths, paths_by_pair, filename="top_paths_interactive.html"):
    initialize()
    G = nx.Graph()

    global_top_paths = all_paths
    for score, path in global_top_paths:
        c, g, d = path
        G.add_node(c, type="chemical", label=data.idx2chemical[c - offsets["chemical"]])
        G.add_node(g, type="gene", label=data.idx2gene[g - offsets["gene"]])
        G.add_node(d, type="disease", label=data.idx2disease[d - offsets["disease"]])



    nodes_by_type = {"chemical": [], "gene": [], "disease": []}
    for n, d in G.nodes(data=True):
        nodes_by_type[d["type"]].append(n)

    max_nodes = max(len(nodes_by_type["chemical"]),
                    len(nodes_by_type["gene"]),
                    len(nodes_by_type["disease"]))
    y_gap = max(1500, 30000 // max_nodes if max_nodes > 0 else 1500)
    x_shift = {"chemical": 0, "gene": 120, "disease": 240}

    pos = {}
```

[۴۷] local_rank

```
for i, n in enumerate(nodes_by_type["chemical"]):
    pos[n] = (x_shift["chemical"], -i * y_gap)
for i, n in enumerate(nodes_by_type["gene"]):
    pos[n] = (x_shift["gene"], -i * y_gap)
for i, n in enumerate(nodes_by_type["disease"]):
    pos[n] = (x_shift["disease"], -i * y_gap)
edge_traces_global = []
for global_rank, (score, path) in enumerate(global_top_paths):
    c, g, d = path
    color = "red" if global_rank < 3 else "rgba(180,180,180,0.25)"
    text_color = "darkred" if global_rank < 3 else "gray"
    for u, v in [(c, g), (g, d)]:
        x0, y0 = pos[u];
        x1, y1 = pos[v]
        xm = (x0 + x1) / 2;
        ym = (y0 + y1) / 2
        edge_traces_global.append(
            go.Scatter(x=[x0, x1], y=[y0, y1], mode="lines",
                       line=dict(width=0.8 + 3 * score, color=color),
                       hoverinfo="skip", showlegend=False)
        )
        edge_traces_global.append(
            go.Scatter(x=[xm], y=[ym + 20], mode="text", text=[f"{score:.3f}"],
                       textfont=dict(size=12, color=text_color),
                       showlegend=False, hoverinfo="skip")
        )

edge_traces_local = []
trace_meta_local = []
```

```
TOP_K_PER_PAIR = 5


for (c, d), paths in paths_by_pair.items():
    if c not in G.nodes() or d not in G.nodes():
        continue
    for score, path, global_rank, local_rank in paths[:TOP_K_PER_PAIR]:
        _, g, _ = path
        if g not in G.nodes():
            continue
        color = "red" if local_rank < 3 else "rgba(180,180,180,0.25)"
        text_color = "darkred" if local_rank < 3 else "gray"
        for u, v in [(c, g), (g, d)]:
            x0, y0 = pos[u];
            x1, y1 = pos[v]
            xm = (x0 + x1) / 2;
            ym = (y0 + y1) / 2
            edge_traces_local.append(
                go.Scatter(x=[x0, x1], y=[y0, y1], mode="lines",
                           line=dict(width=0.8 + 3 * score, color=color),
                           hoverinfo="skip", showlegend=False)
            )
            trace_meta_local.append((c, d))
            edge_traces_local.append(
                go.Scatter(x=[xm], y=[ym + 20], mode="text", text=[f"{score:.3f}"],
                           textfont=dict(size=12, color=text_color),
                           showlegend=False, hoverinfo="skip")
            )
            trace_meta_local.append((c, d))
```

```
    def node_trace_for_type(node_type, color, size):
        xs, ys, texts, hovers = [], [], [], []
        for n, d in G.nodes(data=True):
            if d["type"] == node_type:
                x, y = pos[n]
                xs.append(x);
                ys.append(y)
                label = d["label"]
                hover = f"<b>{label}</b><br>Type: {node_type}<br>ID: {n}"
                texts.append(label)
                hovers.append(hover)
        return go.Scatter(
            x=xs, y=ys, mode="markers+text", text=texts,
            textposition="middle right", hoverinfo="text", hovertext=hovers,
            marker=dict(size=size, color=color, line=dict(width=1.5, color="black")),
            showlegend=True, name=node_type.capitalize()
        )

    node_traces = [
        node_trace_for_type("chemical", "#8dd3c7", 20),
        node_trace_for_type("gene", "#ffffb3", 16),
        node_trace_for_type("disease", "#fb8072", 20),
    ]

    buttons = []
    # Initially no edges visible
    initial_visibility = [False] * len(edge_traces_global) + [False] * len(edge_traces_local) +
[True] * len(
        node_traces)
```

```
    overview_visibility = [True] * len(edge_traces_global) + [False] * len(edge_traces_local) +
[True] * len(
        node_traces)
    buttons.append(dict(
        label="Overview",
        method="update",
        args=[{"visible": overview_visibility}, {"title": "CEPS Explanation: Overview"}]
    ))

    valid_pairs_local = sorted(set(trace_meta_local))
    for (c, d) in valid_pairs_local:
        visibility = [False] * len(edge_traces_global) + \
                     [(meta == (c, d)) for meta in trace_meta_local] + \
                     [True] * len(node_traces)
        label = f"{data.idx2chemical[c - offsets['chemical']]}, {data.idx2disease[d -
offsets['disease']]}"
        buttons.append(dict(
            label=label,
            method="update",
            args=[{"visible": visibility}, {"title": f"CEPS Explanation: {label}"}]
        ))

    fig = go.Figure(
        data=edge_traces_global + edge_traces_local + node_traces,
        layout=go.Layout(
            title=dict(text="Interactive CEPS Explanation", font=dict(size=20), y=0.98),
            hovermode="closest",
            xaxis=dict(visible=False),
```

```
            yaxis=dict(visible=False),
            margin=dict(l=40, r=40, t=120, b=40),
            paper_bgcolor="white",
            plot_bgcolor="white",
            updatemenus=[dict(
                buttons=buttons,
                direction="down",
                x=0.01,
                y=1.12,
                showactive=True,
                bgcolor="white",
                bordercolor="#ccc",
                font=dict(size=11)
            )]
        )
    )

    for i in range(len(edge_traces_global) + len(edge_traces_local)):
        fig.data[i].visible = False

    fig.update_layout(autosize=True, margin=dict(l=80, r=80, t=120, b=80))

    config = {
        'responsive': True,
        'displaylogo': False,
        'modeBarButtonsToRemove': ['select2d', 'lasso2d', 'zoomIn2d', 'zoomOut2d',
'autoScale2d'],
        'modeBarButtonsToAdd': ['zoom2d', 'pan2d', 'resetScale2d', 'toimage']
    }
```

```
    html_str = fig.to_html(include_plotlyjs=True, config=config)
    css = """
    <style>
    .modebar-btn {
        font-size: 20px !important;
        width: 32px !important;
        height: 32px !important;
    }
    .modebar-btn .icon {
        width: 20px !important;
        height: 20px !important;
    }
    </style>
    """
    html_str = html_str.replace('</head>', css + '</head>')

    with open(filename, "w", encoding="utf-8") as f:
        f.write(html_str)

    webbrowser.open(filename)
```

این تابع با استفاده از Plotly یک گراف تعاملی شامل گره‌ها و یال‌های مسیرها را رسم می‌کند. عملکرد اصلی آن به این صورت است:

- یک گراف networkx ساخته می‌شود و تمام گره‌های موجود در مسیرها (بدون محدودیت تعداد) به آن اضافه می‌گردند.

- موقعیت گره‌ها به‌صورت دسته‌ای (داروها در ستون اول، ژن‌ها در ستون دوم، بیماری‌ها در ستون سوم) و با فاصله عمودی متناسب چیده می‌شود.
- یال‌ها با رنگ و ضخامت متناسب با امتیاز رسم می‌شوند (قرمز برای سه مسیر برتر، خاکستری کمرنگ برای بقیه).
- یک منوی کشویی [۴۸]تعبیه می‌شود که کاربر می‌تواند نمای کلی (همه مسیرها) یا مسیرهای مربوط به یک جفت دارو–بیماری خاص را انتخاب کند.
- خروجی به‌صورت یک فایل HTML ذخیره شده و در مرورگر باز می‌شود.

```
def rank_pairs(paths_by_pair, mode="best"):
    """
    Rank (chemical, disease) pairs by path scores.
    mode: "best" | "mean" | "sum"
    """
    pair_scores = []

    for (c, d), paths in paths_by_pair.items():
        scores = [score for score, *_ in paths]

        if mode == "best":
            s = max(scores)
        elif mode == "mean":
            s = sum(scores) / len(scores)
        elif mode == "sum":
            s = sum(scores)
        else:
            raise ValueError("Unknown mode")
```

[۴۸] dropdown

```
        pair_scores.append((s, (c, d)))

    pair_scores.sort(reverse=True, key=lambda x: x[0])
    return pair_scores
```

این تابع با دریافت ساختار داده‌ای شامل تمام مسیرهای دارو-ژن-بیماری، یک لیست اولویت‌بندی‌شده از جفت‌های (دارو، بیماری) بر اساس امتیاز مسیرها ارائه می‌دهد. بسته به حالت انتخابی (بهترین امتیاز، میانگین، یا مجموع امتیازها)، پژوهشگر می‌تولند دیدگاه متفاوتی از قدرت ارتباط بین هر دارو و بیماری داشته باشد. برای مثال، حالت «بهترین امتیاز» قوی‌ترین شواهد موجود را نشان می‌دهد، در حالی که «میانگین» تصویری کلی‌تر از کیفیت تمام مسیرهای مرتبط با یک جفت ارائه می‌کند. خروجی این تابع به‌صورت یک لیست مرتب‌شده از زوج‌های دارو-بیماری به همراه امتیاز نهایی است که می‌تواند برای تصمیم‌گیری در مورد اولویت بررسی‌های آزمایشگاهی یا تحلیل‌های تکمیلی مورد استفاده قرار گیرد. به این ترتیب، پژوهشگر به جای مرور صدها مسیر، می‌تواند مستقیماً بر مهم‌ترین و مستندترین ارتباطات تمرکز کند.

۴.۲ .تفسیر خروجی:

ماژول CEPS.py به‌عنوان سرویس‌دهنده توابع کمکی برای main.py طراحی شده است. وظیفه اصلی آن بارگذاری مدل آموزش‌دیده، محاسبه بازنمایی نهایی گره‌ها و ارائه توابع مورد نیاز برای امتیازدهی، مصورسازی و رتبه‌بندی مسیرها است. در فراخوانی‌های معمول، ابتدا تابع initialize() فراخوانی می‌شود تا گراف و مدل بارگذاری شده و بازنمایی گره‌ها (z) محاسبه گردد. سپس توابعی مانند link_score برای محاسبه شباهت بین گره‌ها، visualize_paths برای ترسیم گراف تعاملی، و rank_pairs برای رتبه‌بندی جفت‌های (دارو، بیماری) بر اساس امتیاز مسیرها مورد استفاده قرار می‌گیرند.

هنگام اجرای مستقیم ماژول(با "__main__" == __name__ if) ، تنها تابع initialize() اجرا شده و پیام تأیید مقداردهی اولیه چاپ می‌شود. خروجی اصلی این ماژول، فایل HTML تعاملی است که پس از فراخوانی visualize_paths در main.py تولید می‌گردد و شامل تمام مسیرهای توضیح‌داده‌شده

برای جفت‌های مورد نظر است. همچنین تابع rank_pairs امکان اولویت‌بندی جفت‌های دارو–بیماری را برای تحلیل‌های بعدی فراهم می‌کند.

۵. ماژول index_lookup.py :جستجوی نام‌ها و تبدیل به ایندکس

این ماژول به‌عنوان یک ابزار کمکی ساده اما کاربردی طراحی شده است تا پژوهشگران بتوانند بدون نیاز به دانستن ایندکس‌های عددی گره‌ها، نام دارو یا بیماری مورد نظر خود را به ایندکس متناظر در گراف تبدیل کنند. این کار به ویژه در مرحله انتخاب نمونه برای توضیح‌دهی یا تحلیل‌های بعدی بسیار مفید است.

۵.۱. کد ماژول:

```
import CEPS

def initialize_if_needed():
    if CEPS.z is None:
        CEPS.initialize()
def get_chemical_index(chemical_name):
    initialize_if_needed()
    for local_idx, name in CEPS.data.idx2chemical.items():
        if name == chemical_name:
            return local_idx + CEPS.offsets["chemical"]
    raise ValueError(f"Chemical '{chemical_name}' not found")

def get_disease_index(disease_name):
    initialize_if_needed()
    for local_idx, name in CEPS.data.idx2disease.items():
        if name == disease_name:
            return local_idx + CEPS.offsets["disease"]
    raise ValueError(f"Disease '{disease_name}' not found")
```

ماژول index_lookup.py با وارد کردن ماژول اصلی CEPS و استفاده از دیکشنری‌های idx2chemical و idx2disease که در فایل hetero_graph.pt ذخیره شده‌اند، امکان جستجوی نام داروها و بیماری‌ها را فراهم می‌کند. تابع initialize_if_needed ابتدا بررسی می‌کند که آیا بازنمایی گره‌ها (z) محاسبه شده است یا خیر؛ در صورت عدم مقداردهی، تابع CEPS.initialize() فراخوانی می‌شود تا گراف و مدل بارگذاری گردند. سپس دو تابع اصلی get_chemical_index و get_disease_index با دریافت نام دارو یا بیماری به‌عنوان رشته، در دیکشنری مربوطه جستجو کرده و در صورت یافتن، ایندکس سراسری آن گره را با اضافه کردن آفست مربوط به نوع گره بازمی‌گردانند. اگر نام مورد نظر یافت نشود، خطای ValueError با پیغام مناسب صادر می‌شود.

۵.۲. تفسیر خروجی:

این ماژول به‌صورت مستقیم اجرا نمی‌شود، بلکه توابع آن در main.py (و سایر ماژول‌ها) فراخوانی می‌گردند. خروجی اصلی آن، ایندکس عددی سراسری گره‌ای است که با نام ورودی (دارو یا بیماری) مطابقت دارد. در صورت یافتن، ایندکس (به‌صورت عدد صحیح) بازگردانده می‌شود و در غیر این‌صورت خطای ValueErrorبا پیغام «Chemical '...' not found» یا «Disease '...' not found» صادر می‌گردد. هنگام اجرای مستقیم ماژول، هیچ خروجی‌ای تولید نمی‌شود، زیرا فاقد بلوک if __name__ == "__main__" است.

۶ .ماژول main.py: یکپارچه‌سازی و اجرای نهایی

نقطه ورود اصلی پروژه، ماژول main.py است که تمام مراحل قبلی را به‌صورت زنجیره‌ای فراخوانی می‌کند: مقداردهیCEPS ، انتخاب یک جفت دارو–بیماری نمونه، اجرای توضیح‌دهنده برای آن جفت، ذخیره مسیرهای توضیح‌داده‌شده و در نهایت مصورسازی آن مسیرها.

۶.۱. کد ماژول:

```
from GNN_Explainer import explain_and_enumerate_paths
import CEPS
import csv
import torch
import index_lookup
```

در ایـــن بخـــش، تـــابع explain_and_enumerate_paths از مـــاژول GNN_Explainer بـــرای اجـــرای فرآینـــد توضـــیح‌پذیری فراخـــوانی می‌شـــود.مـــاژول CEPS شـــامل ســـاختار گـــراف، داده‌هـــا و توابـــع کمکـــی پـــروژه اســـت.کتابخانـــه csv بـــرای ذخیـــره مســـیرها در قالـــب فایـــل جـــدولی اســـتفاده می‌شـــود و torch جهـــت ذخیره‌سازی نتایج در قالب فایـل بـاینری بـه کـار مـی‌رود.همچنـین index_lookup بـرای تبـدیل نـام دارو یـا بیماری به اندیس عددی متناظر در گراف استفاده می‌شود.

```
TOP_K_PAIRS = 20
TOP_M_PATHS = 5
SPECIFIC_CHEMICAL = "Arsenic"
SPECIFIC_DISEASE = "Osteosarcoma"
```

چهار پارامتر کنترلی در ابتدای فایل تعریف شده‌اند:

TOP_K_PAIRS تعداد جفت‌های (دارو، بیماری) برتر برای مصورسازی نهایی را مشخص می‌کند.

TOP_M_PATHSتعداد مسیرهای منتخب برای هر جفت را تعیین می‌نماید.

SPECIFIC_CHEMICALو SPECIFIC_DISEASE امکان تعریف یک جفت خاص جهت تحلیل موردی و مصورسازی جداگانه را فراهم می‌کنند.

```
def main():

    CEPS.initialize()


    src = CEPS.chemical_nodes[0]
    dst = CEPS.disease_nodes[0]


    print(
        "[INFO] Explaining:",
        CEPS.data.idx2chemical[src - CEPS.offsets["chemical"]],
        "→",
        CEPS.data.idx2disease[dst - CEPS.offsets["disease"]],
    )
```

بـــا فراخـــوانی()CEPS.initialize ، گـــراف نـــاهمگن شـــامل گره‌هـــای دارو، ژن و بیمـــاری ســـاخته شـــده و embeddingهـــا، مـــاتریس مجـــاورت و ســـایر ســـاختارهای مـــورد نیـــاز مـــدل آمـــاده می‌شـــوند.پس ابتـــدا مـــاژول CEPSمقـــداردهی می‌شـــود. ایـــن انتخـــاب بـــه‌عنوان یـــک جفـــت نمونـــه (seed pair) بـــرای اجـــرای اولیـــه الگوریتم توضیح‌پذیری در نظر گرفته شـده اسـت. نـام واقعـی ایـن گره‌هـا بـا اسـتفاده از نگاشـت‌های انـدیس به نام چاپ می‌شود تا خروجی برای کاربر قابل‌تفسیر باشد.

```
result = explain_and_enumerate_paths(
    z=CEPS.z,
    edge_index=CEPS.edge_index,
    src=src,
    dst=dst,
    adj=CEPS.adj,
    chemical_nodes=CEPS.chemical_nodes,
    gene_nodes=CEPS.gene_nodes,
    disease_nodes=CEPS.disease_nodes,
    link_score=CEPS.link_score,
    node_ratio=0.4,
    epochs=150,
)

paths = result["paths"]
print(f"[INFO] Paths found after explanation: {len(paths)}")

torch.save(
    {
        "src": src,
        "dst": dst,
        "paths": paths,
        "important_nodes": result["important_nodes"],
```

```
    },
    "explanation_result.pt",
)

with open("explained_paths.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["score", "chemical", "gene", "disease"])
    for score, (c, g, d) in paths:
        writer.writerow(
            [
                score,
                CEPS.data.idx2chemical[c - CEPS.offsets["chemical"]],
                CEPS.data.idx2gene[g - CEPS.offsets["gene"]],
                CEPS.data.idx2disease[d - CEPS.offsets["disease"]],
            ]
        )

print("[INFO] Explained paths saved ✅")
```

تابع توضیح‌دهنده با پارامترهای تعیین‌شده فراخوانی می‌شود. در این فراخوانی، پارامتر node_ratio نسبت گره‌هایی را مشخص می‌کند که در زیرگراف توضیحی حفظ می‌شوند و پارامتر epochs تعداد تکرارهای فرآیند بهینه‌سازی توضیح‌دهنده را تعیین می‌نماید. خروجی این مرحله شامل مسیرهای استخراج‌شده و همچنین مجموعه‌ای از گره‌های دارای اهمیت بالا در زیرگراف توضیحی است. این نتایج در قالب یک فایل باینری با نام explanation_result.pt ذخیره می‌شوند تا امکان استفاده مجدد و تحلیل‌های تکمیلی بدون نیاز به اجرای دوباره مدل فراهم گردد.

```
with open("explained_paths.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["score", "chemical", "gene", "disease"])
```

```
    for score, (c, g, d) in paths:
        writer.writerow([
            score,
            CEPS.data.idx2chemical[c - CEPS.offsets["chemical"]],
            CEPS.data.idx2gene[g - CEPS.offsets["gene"]],
            CEPS.data.idx2disease[d - CEPS.offsets["disease"]],
        ])

print("[INFO] Explained paths saved to explained_paths.csv ✅ ")
```

مسیرهای توضیح‌داده‌شده بـه همـراه نام‌هـای واقعـی (نـه ایـندکس) در یـک فایـل CSV ذخیـره می‌شـوند تـا پژوهشگران بتوانند به‌راحتی آن‌ها را مرور کنند.

```
CEPS.all_paths = paths
CEPS.paths_by_pair = {}
```

در ایــن مرحلــه، مســیرهای استخراج‌شــده در متغیــر CEPS.all_paths ذخیــره می‌شــوند و یــک ســاختار دیکشــنری بــه نــام paths_by_pair بــرای گروه‌بنــدی مســیرها بــر اســاس جفت‌هــای (دارو، بیمــاری) ایجــاد می‌گردد.

```
for global_rank, (score, path) in enumerate(paths):
c, g, d = path
pair = (c, d)
CEPS.paths_by_pair.setdefault(pair, []).append(
(score, path, global_rank)
)
```

در ادامــه، هــر مســیر بــه همــراه امتیــاز خــود پیمــایش شــده و یــک رتبــه سراســری [۴۹]دریافــت می‌کنــد کــه جایگــاه آن مســیر را در کــل مجموعــه مســیرهای استخراج‌شــده نشــان می‌دهــد. ســپس مســیرها بــر اســاس جفت متناظر خود در ساختار paths_by_pair گروه‌بندی می‌شوند.

[۴۹] global rank

```
for pair in CEPS.paths_by_pair:
CEPS.paths_by_pair[pair].sort(reverse=True, key=lambda x: x[0])
CEPS.paths_by_pair[pair] = [
(score, path, global_rank, local_rank)
for local_rank, (score, path, global_rank)
in enumerate(CEPS.paths_by_pair[pair])
]
```

پس از گروه‌بندی، مسیرهای هر جفت بر اساس امتیاز به‌صورت نزولی مرتب می‌شوند و سپس یک رتبه محلی [۵۰] به هر مسیر اختصاص داده می‌شود که نشان‌دهنده جایگاه آن در میان مسیرهای همان جفت است. به این ترتیب هر مسیر دارای دو سطح رتبه‌بندی سراسری و محلی خواهد بود.

```
print("[INFO] Explained pairs:", len(CEPS.paths_by_pair))
print("[INFO] Explained paths:", len(paths))
```

در پایان این مرحله، تعداد جفت‌های دارای مسیر توضیح‌پذیر و تعداد کل مسیرهای استخراج‌شده در خروجی برنامه نمایش داده می‌شود.

```
if SPECIFIC_CHEMICAL and SPECIFIC_DISEASE:
c = index_lookup.get_chemical_index(SPECIFIC_CHEMICAL)
d = index_lookup.get_disease_index(SPECIFIC_DISEASE)

pair = (c, d)

if pair not in CEPS.paths_by_pair:
print("[WARN] No explainer-valid paths for this pair.")
else:
pair_paths = CEPS.paths_by_pair[pair][:TOP_M_PATHS]

pair_paths_by_pair = {pair: pair_paths}
pair_all_paths = [(score, path) for score, path, _, _ in pair_paths]

CEPS.visualize_paths(
all_paths=pair_all_paths,
paths_by_pair=pair_paths_by_pair,
filename="pair_specific_paths.html",
)
```

[۵۰] local rank

```
ranked_pairs = CEPS.rank_pairs(CEPS.paths_by_pair, mode="best")
selected_pairs = [pair for _, pair in ranked_pairs[:TOP_K_PAIRS]]
```

در این مرحله، در صورتی که مقادیر SPECIFIC_CHEMICAL و SPECIFIC_DISEASE تعریف شده باشند، تحلیل یک جفت خاص به‌صورت شرطی فعال می‌شود. ابتدا نام دارو و بیماری به اندیس عددی متناظر در گراف تبدیل می‌شود و در صورت وجود مسیر توضیح‌پذیر برای آن جفت، تنها TOP_M_PATHS مسیر با بالاترین امتیاز انتخاب می‌گردند. این مسیرها به تابع مصورسازی ارسال شده و خروجی در فایل pair_specific_paths.html ذخیره می‌شود که نمایش تعاملی مسیرهای منتخب آن جفت خاص را فراهم می‌کند.

پس از تحلیل جفت مشخص، جفت‌های (دارو، بیماری) بر اساس بیشترین امتیاز مسیر مرتبط با آن‌ها رتبه‌بندی می‌شوند (mode="best"). سپس تعداد TOP_K_PAIRS جفت برتر برای تحلیل نهایی انتخاب می‌گردند.

```
filtered_paths_by_pair = {}
filtered_all_paths = []

for pair in selected_pairs:
top_paths = CEPS.paths_by_pair[pair][:TOP_M_PATHS]
filtered_paths_by_pair[pair] = top_paths
for score, path, _, _ in top_paths:
filtered_all_paths.append((score, path))

print(
f"[INFO] Visualizing {len(filtered_paths_by_pair)} pairs "
f"({TOP_M_PATHS} paths per pair)"
)

CEPS.visualize_paths(
all_paths=filtered_all_paths,
paths_by_pair=filtered_paths_by_pair,
filename="top_pairs_paths.html",
)
```

در ادامه، برای هر یک از جفت‌های منتخب، تنها TOP_M_PATHS مسیر با بیشترین امتیاز نگه‌داشته می‌شود. این مسیرها در دو ساختار مجزا ذخیره می‌شوند: یکی برای نگهداری مسیرهای کلی و دیگری برای نگهداری مسیرهای تفکیک‌شده بر اساس جفت‌ها. در نهایت، تابع مصورسازی فراخوانی شده و خروجی در فایل top_pairs_paths.html ذخیره می‌شود که شامل نمایش تعاملی مسیرهای منتخب برای جفت‌های برتر است.

۶.۲ .تفسیر خروجی:

پس از اجرای main.py ، چندین خروجی متنوع تولید می‌شود که هر یک هدف مشخصی را دنبال می‌کنند. در کنسول، گزارشی از مراحل مقداردهی، جفت نمونه انتخابی، روند بهینه‌سازی توضیح‌دهنده و در نهایت تعداد کل مسیرها و جفت‌های (دارو، بیماری) یافت‌شده نمایش داده می‌شود که صحت انجام فرایند را تأیید می‌کند. هم‌زمان، یک فایل باینری با نام explanation_result.pt شامل اندیس گره‌های مبدأ و مقصد، لیست کامل مسیرها همراه امتیاز و گره‌های مهم زیرگراف توضیحی ذخیره می‌گردد که برای بارگذاری مجدد و تحلیل‌های پیشرفته در محیط پایتون قابل استفاده است. همچنین فایل explained_paths.csv با ستون‌های امتیاز، نام دارو، نام ژن و نام بیماری تولید می‌شود که پژوهشگران می‌توانند آن را در نرم‌افزارهای صفحه‌گسترده باز کرده و به جستجو، فیلتر و مقایسه مسیرها بپردازند.

در بخش مصورسازی، بسته به تنظیمات پارامترها، یک یا دو صفحه HTML تعاملی ایجاد می‌شود. اگر دارو و بیماری خاصی تعریف شده باشد، فایل pair_specific_paths.html تنها مسیرهای مرتبط با آن جفت را به صورت گراف تعاملی نمایش می‌دهد و امکان بررسی عمیق مکانیسم اثر آن مورد خاص را فراهم می‌کند. علاوه بر این، فایل top_pairs_paths.html برای TOP_K_PAIRS جفت برتر بر اساس امتیاز مسیرها، همراه با TOP_M_PATHS مسیر با بالاترین امتیاز برای هر جفت، ترسیم می‌شود. این نمای کلی از مهم‌ترین ارتباطات دارو–ژن–بیماری، پژوهشگر را در شناسایی الگوهای مشترک و تولید فرضیات جدید درمانی یاری می‌دهد.

تحلیل نتایج برای پزشکان و پژوهشگران

در بخش‌های پیشین، برای هر ماژول تفسیر خروجی مربوط به آن ارائه شد. در این بخش، تحلیلی کلی از نتایج نهایی پروژه از منظر عملکرد مدل، مسیرهای استخراج‌شده و کاربرد آن‌ها برای پزشکان و پژوهشگران ارائه می‌گردد.

پس از اجرای کامل فرآیند ها ، مدل نهایی با AUC ۰.۸۷ توانایی قابل قبولی در پیش‌بینی روابط جدید دارو–بیماری نشان داد. برای نمونه موردی داروی Ciprofloxacin و بیماری Carcinoma، از میان ۱۵ مسیر استخراج‌شده با امتیاز بالای ۰.۹، ژن‌های TP53، EGFR و PTEN به‌عنوان پرتکرارترین واسطها ظاهر شدند که همگی در پایگاه‌های معتبر مانند NCBI و PubMed با سرطان مرتبط شناخته می‌شوند.

پس از اجرای کامل پروژه، سه خروجی اصلی تولید می‌شود که هدف آنها ارائه شواهد و مدارک لازم برای توجیه پیشنهاد دارویی مدل است. این خروجی‌ها به‌گونه‌ای طراحی شده‌اند که پژوهشگر یا پزشک بتواند بدون نیاز به دانش فنی عمیق در مورد شبکه‌های عصبی، ارتباطات پیشنهادی را بررسی کرده و در صورت نیاز، آزمایش‌های تجربی مناسب طراحی کند. در این راهنما، ابتدا هر یک از این خروجی‌ها را معرفی کرده و سپس گام‌های عملی برای تفسیر و بهره‌برداری از آنها را شرح می‌دهیم.

خروجی اول: فایل CSV شامل فهرست مسیرهای دارو–ژن–بیماری

پس از اجرای main.py، فایلی با نام explained_paths.csv در پوشه پروژه ایجاد می‌شود. این فایل یک جدول ساده شامل تمام مسیرهایی است که مدل برای یک جفت دارو-بیماری خاص (مثلاً داروی Ciprofloxacin و بیماری Carcinoma)به‌عنوان مسیرهای توضیح‌دهنده شناسایی کرده است. هر سطر از این جدول یک مسیر سه‌گامی دارو ← ژن ← بیماری را نشان می‌دهد. ستون‌های این فایل عبارتند از: امتیاز [۵۱]که بین صفر و یک متغیر است و هر چه به یک نزدیک‌تر باشد، نشان‌دهنده اطمینان بیشتر مدل به آن مسیر است. این امتیاز از جمع دو شباهت کسینوسی بین بردار بازنمایی دارو و ژن و بین بردار بازنمایی ژن و بیماری به‌دست می‌آید و عملاً معیاری برای قوت ارتباط در نظر

[۵۱] score

گرفته می‌شود. ستون بعدی نام دارو است که بر اساس شناسه‌های پایگاه CTD درج شده است. سپس ستون ژن با نماد استاندارد ژن(مثلاًTP53 ، EGFRیا BRCA1) و در نهایت ستون بیماری با نام استاندارد شده بر اساس آنتولوژیMONDO .

برای استفاده از این فایل، پژوهشگر می‌تواند آن را در نرم‌افزار صفحه‌گسترده‌ای مانند Excel باز کرده و بر اساس ستون امتیاز، مسیرها را به ترتیب نزولی مرتب کند. مسیرهایی که در بالای لیست قرار می‌گیرند، قوی‌ترین شواهد را برای ارتباط دارو و بیماری فراهم می‌کنند. ژن‌هایی که در چندین مسیر با امتیاز بالا تکرار شده‌اند، به‌عنوان کاندیدهای اصلی برای نقش میانجی بین دارو و بیماری مطرح می‌شوند. برای مثال، اگر ژن TP53 در چندین مسیر با امتیاز بالای ۰.۹ ظاهر شود، این ژن می‌تواند هدف مناسبی برای بررسی‌های بیشتر باشد. پژوهشگر می‌تواند نام این ژن‌ها را در پایگاه‌های داده‌ای مانند NCBI Gene یا PubMed جستجو کند و ببیند آیا پیش‌تر ارتباط آنها با بیماری هدف یا تعامل با داروی مورد نظر در مقالات علمی گزارش شده است یا خیر. این تطابق با ادبیات علمی می‌تواند اعتماد به پیش‌بینی مدل را افزایش دهد.

خروجی دوم: گراف تعاملیHTML

دومین خروجی که مهم‌ترین و بصری‌ترین بخش پروژه است، شامل یک یا دو فایل HTML تعاملی می‌باشد. این فایل‌ها با استفاده از کتابخانه Plotly ساخته شده و مسیرهای شناسایی‌شده را به‌صورت یک گراف قابل کاوش نمایش می‌دهند .

-اگر پارامترهای SPECIFIC_CHEMICAL و SPECIFIC_DISEASE در ابتدای فایل main.py تعریف شده باشند، فایل pair_specific_paths.html ایجاد می‌شود که تنها مسیرهای مربوط به آن جفت خاص را نشان می‌دهد .

-همچنین فایل top_pairs_paths.html برای TOP_K_PAIRS جفت برتر (بر اساس امتیاز مسیرها) به همراه TOP_M_PATHS مسیر با بالاترین امتیاز برای هر جفت تولید می‌گردد .

در این گراف‌ها، گره‌ها به سه دسته تقسیم شده و در سه ستون عمودی قرار گرفته‌اند: داروها در ستون نخست با رنگ سبز، ژن‌ها در ستون میانی با رنگ زرد و بیماری‌ها در ستون سوم با رنگ قرمز. این چیدمان به بیننده امکان می‌دهد تا در یک نگاه، سلسله‌مراتب مسیرها و نقش هر گره را درک

کند. یال‌هایی که داروها را به ژن‌ها و ژن‌ها را به بیماری‌ها متصل می‌کنند، با ضخامتی متناسب با امتیازشان رسم شده‌اند. سه مسیر برتر از نظر امتیاز با رنگ قرمز پررنگ مشخص می‌شوند تا سریعاً قابل شناسایی باشند و بقیه مسیرها با رنگ خاکستری کمرنگ نمایش داده می‌شوند تا از شلوغی بصری کاسته شود .

با قرار دادن نشانگر ماوس روی هر گره، نام کامل و نوع آن نمایش داده می‌شود و با نگه‌داشتن ماوس روی هر یال، امتیاز آن یال قابل مشاهده است. این ویژگی‌های تعاملی به پزشک یا پژوهشگر امکان می‌دهد تا به‌صورت شهودی گراف را کاوش کند، ژن‌های واسط را شناسایی کند و روابط پیچیده را بهتر درک نماید. برای استفاده از این گراف‌ها، کافی است فایل HTML را در یک مرورگر وب باز کنید. هیچ نیاز به نصب نرم‌افزار اضافی نیست. می‌توانید با کلیک و کشیدن، گراف را جابه‌جا کرده و با اسکرول کردن، بزرگنمایی را تغییر دهید. این ابزار به‌ویژه برای ارائه در جلسات یا انتشار در کنار مقالات علمی بسیار مفید است.

خروجی سوم: فایل checkpoint برای تحلیل‌های بعدی

سومین خروجی با نام explanation_result.pt یک فایل باینری است که حاوی تمام اطلاعات خام مربوط به فرایند توضیح‌دهی می‌باشد. این فایل شامل ایندکس گره‌های مهم، ماسک اهمیت یال‌ها و لیست کامل مسیرها به همراه امتیازها است. اگر پژوهشگر بخواهد بعداً تحلیل دیگری با پارامترهای متفاوت انجام دهد (مثلاً تغییر نسبت گره‌های مهم یا تنظیم مجدد ضریب‌های جریمه)، می‌تواند این فایل را بارگذاری کرده و بدون نیاز به اجرای مجدد مدل سنگین اصلی، مسیرها را دوباره پردازش کند. این فایل بیشتر برای استفاده‌های فنی و توسعه‌دهندگان کاربرد دارد، اما وجود آن برای بازتولید نتایج و شفافیت علمی ضروری است.

گام‌های عملی برای تفسیر و اعتبارسنجی بالینی به صورت زیر است :

پس از دریافت این سه خروجی، پژوهشگر می‌تواند مراحل زیر را برای استفاده حداکثری از نتایج دنبال کند:

ابتدا با باز کردن فایلCSV ، ژن‌هایی را که در چندین مسیر با امتیاز بالا ظاهر شده‌اند شناسایی کند. این ژن‌ها کاندیدهای اصلی برای نقش میانجی هستند. سپس با مراجعه به پایگاه‌های داده‌ای مانند

PubMed، NCBI Gene یا STRING ، جستجویی درباره ارتباط آن ژن با بیماری هدف انجام دهد. اگر مقالات متعددی این ارتباط را تأیید کرده باشند، اعتماد به مسیر پیشنهادی افزایش می‌یابد. همچنین می‌تواند تعامل دارو با آن ژن را در پایگاه‌هایی مانند DrugBank بررسی کند.

در گام بعدی، گراف تعاملی را باز کرده و مسیرهای برتر را به‌صورت بصری مشاهده کند. آیا ژن‌های کلیدی که در CSV شناسایی شده‌اند، در گراف نیز با یال‌های پررنگ به دارو و بیماری متصل هستند؟ آیا ساختار کلی گراف با دانش قبلی پژوهشگر درباره مکانیسم بیماری همخوانی دارد؟ برای مثال، در سرطان پستان، آیا مسیرهایی شامل ژن‌های مرتبط با گیرنده استروژن (ESR1) یا HER2 دیده می‌شود؟

پس از تأیید اولیه، پژوهشگر می‌تواند مسیرهای کاملاً جدید اما با امتیاز بالا را برای طراحی آزمایش‌های تجربی انتخاب کند. برای نمونه، اگر ژنی مانند ABCA1 که تاکنون ارتباط آن با سرطان ریه کمتر بررسی شده، در مسیرهای برتر ظاهر شود، می‌توان آزمایشی طراحی کرد که در آن بیان این ژن در سلول‌های سرطانی ریه پس از تیمار با داروی پیشنهادی اندازه‌گیری شود. همچنین می‌توان با تکنیک‌هایی مانند CRISPR-Cas9 آن ژن را خاموش کرده و تغییر حساسیت سلول‌ها به دارو را بررسی کرد.

در نهایت، پژوهشگر می‌تواند نتایج خود را با سایر منابع داده‌های بزرگ مانند TCGA یا GEO مقایسه کند تا ببیند آیا الگوهای بیانی ژن‌های پیشنهادی در نمونه‌های بیماران با پیش‌بینی مدل همخوانی دارد یا خیر.

هدف نهایی این است که مدل به‌عنوان یک ابزار فرضیه‌ساز در کنار پژوهشگر قرار گیرد و با ارائه شواهد مستند، مسیر را برای اکتشافات جدید هموار کند. خروجی‌های این پروژه دقیقاً به همین منظور طراحی شده‌اند تا پل ارتباطی بین محاسبات و آزمایشگاه را به شکلی مؤثر و قابل فهم برقرار سازند.

جمع بندی و نتیجه‌گیری

هدف اصلی این پروژه فراتر از پیش‌بینی صرف ارتباطات جدید دارو-بیماری بود. آنچه که کار حاضر را از مدل‌های پایه‌ای مانند R-GCN و Decagon متمایز می‌کند، قابلیت توضیح‌پذیری آن است. در حالی که آن مدل‌ها صرفاً یک امتیاز یا احتمال برای هر جفت دارو-بیماری تولید می‌کنند و عملاً به‌صورت جعبه‌سیاه عمل می‌کنند، در این پروژه با ترکیب GNN Explainer و ماژول CEPS ، توانسته‌ایم زیرگراف‌هایی را استخراج کنیم که نشان می‌دهند مدل بر اساس کدام مسیرهای زیستی (دارو ← ژن ← بیماری) به پیش‌بینی خود رسیده است. به عبارت دیگر، مدل نه تنها نتیجه را اعلام می‌کند، بلکه مدارک و شواهد خود را نیز به‌صورت شفاف در اختیار پزشک یا پژوهشگر قرار می‌دهد.

این قابلیت در کاربردهای بالینی اهمیت حیاتی دارد. یک پزشک نمی‌تواند صرفاً بر اساس یک عدد، درمانی را تجویز کند یا هزینهٔ آزمایش‌های گرانقیمت را توجیه نماید. اما وقتی مدل بتواند بگوید «این دارو را برای این بیماری پیشنهاد می‌کنم زیرا از طریق ژن‌های TP53 ، EGFR و مسیر PI3K/AKT با بیماری مرتبط است»، آنگاه می‌تواند به‌عنوان یک ابزار تصمیم‌یار مورد اعتماد قرار گیرد. خروجی‌های نهایی پروژه (فایل CSV ، گراف تعاملی HTML و فایل checkpoint)دقیقاً برای همین منظور طراحی شده‌اند تا پل ارتباطی بین محاسبات و آزمایشگاه را به شکلی مؤثر و قابل فهم برقرار سازند.

در نهایت، این پروژه نشان می‌دهد که می‌توان دقت بالای مدل‌های گرافی را با شفافیت لازم برای کاربردهای پزشکی ترکیب کرد. گام بعدی می‌تواند اعتبارسنجی تجربی تعدادی از مسیرهای پیشنهادی در محیط آزمایشگاهی باشد تا فرضیه‌های تولیدشده توسط مدل به یافته‌های بالینی ملموس تبدیل شوند.

منابع:

[1] M. Zitnik, M. Agrawal, and J. Leskovec, “Modeling polypharmacy side effects with graph convolutional networks,” *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018, doi: 10.1093/bioinformatics/bty294.

[2] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” in *The Semantic Web – 15th International Conference (ESWC 2018)*, Lecture Notes in Computer Science, vol. 10843, Springer, 2018, pp. 593–607, doi: 10.1007/978-3-319-93417-4_38.